SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

THANJAVUR | KUMBAKONAM | CHENNAI

END-SEMESTER PROJECT REPORT

on

MAT340 – REINFORCEMENT LEARNING & DECISION MAKING

Submitted By

Student Name	:	Abirami B
Register Number	:	127150003
Programme	:	M.Sc. Data Science
Academic Year	:	2026 – 2027
School	:	School of Arts, Sciences, Humanities & Education
Project Title	:	Dataset-Informed Reinforcement Learning for Grid-Based Autonomous Navigation: A Performance Comparison of Q-Learning, SARSA, DQN and PPO

Internal Faculty Signature

External Faculty Signature(s)

Contents

List of Figures 3
List of Tables 4
Abstract 5
1 Introduction 6
2 Problem Statement 9
3 Objectives 12
4 Dataset and Environment Description 13
5 Methodology 17
6 Experimental Setup 21
7 Evaluation Metrics 24
8 Results and Analysis 26
9 Sample Predictions and Navigation Behaviour 33
10 Discussion 35
11 Limitations 37
12 Future Enhancements 39
Conclusion 40
Bibliography 41
A Source Code 42

List of Figures

Figure 4.1: Dataset-informed simulated 8 × 8 navigation map generated by the notebook 15
Figure 5.1: Workflow of the dataset-informed comparative study 17
Figure 8.1: Training reward of seed 0 for each algorithm, including raw episode reward and moving average 26
Figure 8.2: Smoothed training reward, mean ± standard deviation over three seeds 27
Figure 8.3: Smoothed training success rate, mean ± standard deviation over three seeds 27
Figure 8.4: Greedy evaluation metrics for the four algorithms; error bars show standard deviation across seeds 29
Figure 8.5: Mean evaluation success rate against mean training time; error bars show seed standard deviation 30
Figure 8.6: Success-rate comparison highlighting the highest mean result, SARSA 31
Figure 9.1: Greedy paths of the seed-0 agents with slip noise disabled for illustration 33

List of Tables

Table 1.1: Overview of the four reinforcement learning algorithms 7
Table 2.1: Reward structure of the navigation environment 10
Table 3.1: Objectives and the evidence supporting them 12
Table 4.1: Summary statistics of the numeric columns of the dataset 13
Table 4.2: Categorical columns of the dataset 13
Table 4.3: Reinforcement-learning components identified in the dataset 14
Table 4.4: Weather categories and assigned slip probabilities 14
Table 4.5: Configuration of the dataset-informed navigation environment 15
Table 6.1: Software environment and execution platform 21
Table 6.2: Settings shared by all algorithms 21
Table 6.3: Hyperparameters of the four algorithms 22
Table 7.1: Evaluation metrics used in this study 24
Table 8.1: Number of training episodes completed in 60,000 environment steps 26
Table 8.2: Greedy evaluation results for each trained agent 28
Table 8.3: Evaluation metrics, mean ± population standard deviation over seeds 28
Table 8.4: Median evaluation results over the three seeds (derived from Table 8.2) 29
Table 8.5: Algorithm ranking using mean success rate and tie-break metrics 29
Table 8.6: Convergence, stability and training time 30
Table 9.1: Routes of the seed-0 greedy agents without slip noise (read from Figure 9.1) 33
Table 9.2: Outcome fractions for the trained agents 34

Abstract

This project compares four reinforcement learning algorithms—Q-Learning, SARSA, Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—on a dataset-informed simulated autonomous-navigation task. The Kaggle Autonomous Navigation Control Dataset contains 1,000 records and 16 columns. It does not provide reward, next-state, termination or episode identifiers and therefore cannot serve as a complete logged reinforcement-learning dataset. However, two of its fields are used to inform the simulator: the proportion of records with Distance_to_Obstacle_m below an assumed 15 m threshold determines obstacle density, while Weather_Condition determines an episode-level action-slip probability through an explicitly assumed weather-to-slip mapping.

The environment is an 8 × 8 grid with a fixed map generated using seed 123, seven obstacle cells, a start cell at (0, 0), a goal cell at (7, 7), four discrete movement actions and a 100-step episode limit. The observed close-obstacle fraction is 0.155, which is used as the obstacle density. The mean of the weather-mapped slip values is 0.107. The reward design assigns +100 for reaching the goal, −50 for collision, −3 for moving beyond the grid boundary and −1 for an ordinary movement.

Each algorithm was trained for 60,000 environment steps using seeds 0, 1 and 2. Each trained policy was evaluated greedily for 100 episodes on fixed evaluation seeds. The mean success rates were 0.827 ± 0.040 for SARSA, 0.810 ± 0.022 for DQN, 0.580 ± 0.396 for Q-Learning and 0.257 ± 0.363 for PPO. SARSA therefore ranks first by mean success rate, although its lead over DQN is only 0.017 and is smaller than the observed seed-to-seed variation. The result is indicative rather than statistically conclusive. Q-Learning and SARSA required approximately two seconds per run, PPO approximately 70 seconds and DQN approximately 133 seconds on the recorded CPU execution platform.

All findings are obtained from a controlled simulation. The weather-to-slip mapping is a modelling assumption, not an empirically calibrated physical relationship, and the results do not establish the behaviour of a real vehicle or robot.

Keywords: Reinforcement learning; Autonomous navigation; Dataset-informed simulation; Q-Learning; SARSA; Deep Q-Network; Proximal Policy Optimization.

Chapter 1

Introduction

1.1 Reinforcement Learning and Sequential Decision-Making

Reinforcement learning (RL) studies how an agent can learn to make decisions by interacting with an environment. At each discrete time step t, the agent receives an observation or state s_t, selects an action a_t, receives a scalar reward r_{t+1}, and moves to a successor state s_{t+1}. The policy $\pi(a|s)$ describes how actions are selected. The objective is to maximise the expected discounted return over the sequence of future rewards [1].

The discounted return is the weighted sum of the future rewards:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \ldots = \Sigma_{k=0}^{\infty}\, \gamma^k\, r_{t+k+1} \tag{1.1}$$

Here $\gamma \in [0, 1)$ is the discount factor. Discounting gives immediate rewards greater weight than distant rewards while retaining the influence of long-term outcomes. In navigation, an action that appears locally inefficient may lead to a safer or shorter complete route. This makes sequential reasoning essential.

The interaction is commonly formalised as a Markov decision process (MDP), consisting of a state space, action space, transition probabilities, reward function and discount factor. The Markov property states that the next state and reward depend on the current state and action rather than the complete earlier history. In this project, the map is fixed and the agent position identifies the grid cell; the additional observation fields are deterministic functions of that position. Thus, the cell index is sufficient for the tabular methods.

Two value functions are useful. The state-value function $V^{\pi}(s)$ is the expected return from state s under policy π. The action-value function $Q^{\pi}(s,a)$ is the expected return after selecting action a in state s and then following π. Bellman relationships express these values through immediate reward and discounted successor value. Q-Learning and DQN estimate an optimal action-value function, SARSA estimates the value of its current exploratory policy, and PPO improves a parameterised policy using a policy-gradient objective.

1.2 Autonomous Navigation

Autonomous navigation requires an agent to move from a start location to a target while avoiding obstacles and respecting movement constraints. Conventional navigation systems may combine mapping, localisation, route planning and low-level control. Search algorithms such as breadth-first search and A* can produce routes when a sufficiently accurate map is available. Reinforcement learning instead learns a decision policy from interaction and can incorporate stochastic outcomes directly into the transition model.

The present study uses a discrete grid world as a controlled experimental abstraction. The grid preserves the central decision-making elements—position, obstacles, goal, movement actions, movement cost and uncertain control—while excluding continuous vehicle dynamics, perception pipelines and moving traffic. The simplification makes repeated experiments feasible and allows the four learning methods to be compared under a shared environment.

The abstraction also limits interpretation. The state is a grid cell rather than a physical pose, the action set contains only four cardinal moves, and the obstacle map is fixed. Consequently, the results are

evidence about the algorithms in this particular simulation and not a demonstration of road-ready autonomous driving.

1.3 Motivation for Comparing Algorithms

RL algorithms differ in their learning targets, policy representations, exploration mechanisms and use of collected experience. Q-Learning and SARSA are tabular temporal-difference algorithms. DQN replaces the table with a neural network and uses replay memory and a target network. PPO is an actor-critic method that directly updates a stochastic policy using a clipped objective. Comparing these methods helps illustrate the trade-offs between simplicity, computational cost, stability and performance.

The comparison is organised around four dimensions: the object learned (action values or policy), on-policy versus off-policy updates, tabular versus neural representation, and the way experience is reused. Q-Learning and SARSA use each transition for an immediate table update. DQN stores transitions and samples mini-batches repeatedly. PPO collects short rollouts, performs multiple optimisation epochs and discards the rollout.

1.4 Algorithms Studied

Table 1.1: Overview of the four reinforcement learning algorithms

Algorithm	Family	Policy/update type	Representation in this project
Q-Learning	Tabular value-based	Off-policy	64 × 4 Q-table; one state per grid cell
SARSA	Tabular value-based	On-policy	64 × 4 Q-table; one state per grid cell
DQN	Deep value-based	Off-policy	8–64–64–4 neural network, replay buffer and target network
PPO	Deep actor-critic	On-policy	Stable-Baselines3 MlpPolicy with two 64-unit hidden layers

Q-Learning and SARSA differ principally in their temporal-difference targets. DQN retains the Q-Learning optimality target but uses function approximation and stabilising mechanisms. PPO belongs to a different family: it learns a policy and a value estimate, and restricts the magnitude of each policy update through clipping [2–5].

1.5 Scope and Relation to the Course

The work covers inspection of the Kaggle dataset, derivation of two simulator properties from dataset fields, implementation of a Gymnasium navigation environment, training of four algorithms across three random seeds, greedy evaluation and comparative analysis. The dataset informs obstacle density and episode-level slip probability, but it does not supply reward transitions or an interactive environment. The simulator's reward and transition rules remain explicitly designed in the notebook.

The project applies core topics from MAT340 – Reinforcement Learning & Decision Making. It does not implement a Transformer or attention mechanism. Sequence-modelling approaches such as Decision Transformer are relevant as possible future work, but they were not part of the experiments reported here [6].

1.6 Organisation of the Report

Chapter 2 formalises the navigation task. Chapter 3 lists the objectives. Chapter 4 describes the dataset and dataset-informed environment. Chapter 5 explains the learning methods and experimental workflow. Chapter 6 records the implementation settings, and Chapter 7 defines the evaluation metrics.

Chapter 8 presents measured results, while Chapter 9 examines example navigation behaviour. Chapters 10–12 discuss the findings, limitations and future enhancements. Appendix A reproduces the complete notebook code.

1.7 MDP Formulation, Truncation and Discounting

The grid navigation task can be described using the standard MDP tuple (S, A, P, R, γ). The state set S contains the 64 grid cells, and the action set A contains four cardinal moves. The transition kernel P is stochastic because the intended action may be replaced by a random action according to the slip probability sampled for the current episode. The reward function R is defined by the goal, obstacle, boundary and ordinary movement rules. The discount factor is $\gamma = 0.99$.

The state representation used by the tabular algorithms is a row-major integer index. If the agent is at row r and column c, the state index is $s = 8r + c$. This representation is compact and one-to-one for the fixed map. DQN and PPO instead receive a vector containing normalised position, relative goal offsets and local obstacle indicators. Because the map does not change during the experiment, both representations encode the same underlying agent location, although they present it differently to the learning algorithm.

The discount factor controls how future rewards contribute to current action values. A value close to one gives substantial weight to rewards that occur after a sequence of movements. This is appropriate for a task in which the goal reward is delayed until the agent completes a route. The chosen $\gamma = 0.99$ therefore supports propagation of terminal reward information backwards through temporal-difference updates. The report does not claim that this value is optimal; no hyperparameter search was performed.

The environment distinguishes true termination from time-limit truncation. A goal or collision ends the underlying task, while the 100-step limit ends an episode for practical reasons. In value-based learning, the target is masked only for true terminal events. Retaining the bootstrap term at truncation avoids treating the time limit as though it were an absorbing state with no future value. This distinction is part of the implementation and is relevant when interpreting the learning equations.

Chapter 2

Problem Statement

2.1 The Navigation Task

The agent begins at the top-left grid cell (row 0, column 0) and must reach the bottom-right cell (row 7, column 7) on an 8 × 8 map. Seven cells are obstacles. The map is generated with a fixed random seed and is accepted only if breadth-first search confirms that a path exists. An episode ends when the agent reaches the goal, collides with an obstacle, or reaches the 100-step time limit.

In an obstacle-free 8 × 8 grid, a shortest route from (0,0) to (7,7) requires seven downward moves and seven rightward moves, giving a Manhattan lower bound of 14 actions. Enumerating the down/right-only routes on the generated map shows that 257 distinct 14-move routes avoid all seven obstacle cells (a value derived from the map, not printed by the notebook). The map therefore keeps the minimum of 14 actions while still providing local hazards.

2.2 Observations

The environment returns an eight-dimensional observation vector. Its first four values encode the agent row and column and the row and column offsets to the goal, each divided by 7 (the grid size minus one). The remaining four binary values indicate whether the cells above, below, left and right are blocked by an obstacle or the grid boundary. The observation space is a continuous Box with bounds −1 and +1 and shape (8,).

The tabular methods convert the first two position values back to integer coordinates and flatten them into a state index $s = 8r + c$, where $s \in \{0,\ldots,63\}$. DQN and PPO receive the full eight-dimensional observation. Since the map and goal remain fixed, the sensor indicators and goal offsets are deterministic functions of the current cell. The vector does not add hidden state information in this experiment, but it provides a numeric representation suitable for neural networks.

2.3 Actions and Stochastic Control

The action space is discrete with four actions: 0 = up, 1 = down, 2 = left and 3 = right. At the beginning of each episode, the environment samples a slip probability from the dataset-derived weather mapping. At each step, with probability p_{slip} the selected action is replaced by a uniformly sampled action from the four available actions; otherwise the intended action is executed.

The weather-to-slip mapping is a modelling assumption. The notebook assigns 0.05 to clear or sunny conditions, 0.07 to cloudy or overcast conditions, 0.10 to rainy conditions, 0.12 to foggy conditions and 0.15 to snowy conditions. The observed dataset contains Sunny, Rainy, Foggy and Snowy categories. The resulting mean across dataset rows is 0.107. This mapping is not estimated from measured vehicle-control failures and should not be interpreted as a physical calibration.

The effect of the slip noise can be quantified from these assumed values. Because the replacement action is drawn from all four moves, it equals the intended move one time in four, so the executed move differs from the intended move with probability $0.75\ p_{slip}$. Averaged over the dataset weather mix this is $0.75 \times 0.107 \approx 0.080$. For a shortest route of 14 moves, the probability that every executed move equals the intended move, averaged over the four weather categories, is about 0.34 (derived from the assumed slip values, not measured). Even a good policy must therefore expect deviations from its intended route in many episodes. Whether this is the main reason for the collision rates in Chapter 8 was not tested.

2.4 Rewards and Termination

The reward function assigns +100 for reaching the goal, −50 for entering an obstacle, −3 for attempting to move beyond the grid boundary, and −1 for an ordinary movement. A collision terminates the episode without moving the agent into the obstacle. A boundary action leaves the agent in its current cell and the episode continues. If no terminal event occurs within 100 steps, the episode is truncated.

Table 2.1: Reward structure of the navigation environment

Event	Reward	Episode effect
Reach the goal	+100	Terminates successfully
Move into obstacle	−50	Terminates as collision; position unchanged
Move beyond boundary	−3	Agent remains in place; episode continues
Ordinary movement	−1	Episode continues
100 steps elapsed	—	Truncated as timeout

The reward structure favours goal completion and penalises collisions more strongly than ordinary movement. The per-step cost encourages shorter routes, while the boundary penalty discourages repeated invalid actions. Since timeout is a truncation rather than a true terminal transition, the value-based algorithms continue to bootstrap at the time limit.

2.5 Formal Statement and Intended Outcome

The task is modelled as a finite MDP with stochastic transitions induced by action slip. The objective is to learn a policy π that maximises the expected discounted return from the start state:

$$J(\pi) = E_{\pi}[\ \Sigma_{t=0}^{T}\ \gamma^{t}\ r_{t+1}\ ] \qquad (2.1)$$

where T is the final step of the episode and $\gamma = 0.99$. The experimental objective is comparative: to measure how often each trained greedy policy reaches the goal, how often it collides, how many steps successful episodes require, and how much computation training consumes.

The stochasticity is partly controlled by the episode-level slip probability and partly by the random replacement draw at each step. Episodes associated with snowy conditions use a higher assigned slip probability than episodes associated with sunny conditions. The simulator therefore varies action reliability between episodes while keeping the obstacle map fixed. This is a deliberate design choice and is not evidence that the dataset measured actual control failure.

2.6 Transition Outcomes and Episode Lifecycle

An episode begins when reset() places the agent at the start cell and samples a weather-associated slip value from the dataset-derived array. The current slip value is retained for the episode. At each time step, the agent chooses one of the four discrete actions. The environment then determines whether the action is replaced by a random action and calculates the resulting position.

If the attempted movement crosses the grid boundary, the position remains unchanged and the agent receives −3. If the destination is an obstacle, the episode ends as a collision and the agent receives −50. If the destination is the goal, the episode ends successfully and the agent receives +100. All other valid movements receive −1. The step counter is incremented, and an episode that reaches 100 steps without a terminal event is marked as truncated.

This order of operations matters because the reward and outcome flag correspond to the executed action rather than the intended action. The information dictionary records success and collision explicitly. The

evaluation function uses these flags to compute outcome fractions. As a result, the metrics are tied to simulator events and do not rely on interpreting a reward threshold.

Chapter 3

Objectives

The project objectives are defined to correspond directly to operations and measurements present in the notebook. The distinction between completed implementation tasks and conclusions supported by a small number of seeds is retained.

1. To inspect the Kaggle Autonomous Navigation Control Dataset and identify fields that can inform a navigation simulator.
2. To construct a dataset-informed 8 × 8 grid environment using obstacle proximity and weather category information.
3. To implement or examine Q-Learning, SARSA, DQN and PPO within the same Gymnasium environment.
4. To train each algorithm for a fixed budget of 60,000 environment steps using random seeds 0, 1 and 2.
5. To evaluate each trained policy greedily over 100 fixed evaluation episodes.
6. To compare reward, success rate, successful-route length, collision rate, timeout rate, convergence step, reward variability and training time.
7. To identify the best-performing method using the measured mean success rate while acknowledging seed variation and modelling limitations.

Table 3.1: Objectives and the evidence supporting them

No.	Objective	Evidence in notebook	Status
1	Inspect dataset	1,000 rows, 16 columns; field matching for obstacle distance and weather	Met
2	Dataset-informed simulator	Obstacle density 0.155; episode slip sampled from weather mapping	Met
3	Use four RL methods	Tabular Q-Learning/SARSA, custom PyTorch DQN and Stable-Baselines3 PPO	Met
4	Common training budget	12 full runs; 60,000 steps per run and three seeds	Met
5	Greedy evaluation	100 episodes per trained agent with fixed evaluation seeds	Met
6	Compare recorded metrics	Per-seed and aggregate metrics, charts and ranking generated	Met
7	Identify best method cautiously	SARSA highest mean success; lead over DQN smaller than seed variation	Partly met

The final objective is only partly met in the sense that the ranking is descriptive rather than statistically established. Three seeds provide a limited estimate of variability. The report therefore identifies SARSA as the highest mean result in the recorded experiment without claiming that it is universally superior.

Chapter 4

Dataset and Environment Description

4.1 Dataset Use in Reinforcement Learning

An online RL agent generates transitions by acting in an environment. A conventional transition record contains a state, action, reward, next state and termination indicator. The Kaggle dataset used in this project is a collection of driving-condition records rather than a complete sequence of transitions. It has no reward, next-state, done, episode identifier or time-step field. It therefore cannot independently serve as the interactive environment required by the four online algorithms.

The updated notebook uses the dataset in a narrower and explicit way. It derives an obstacle-density statistic from the continuous Distance_to_Obstacle_m field and maps each Weather_Condition category to a stipulated action-slip probability. The simulator itself still defines the grid transitions and reward function. This is best described as a dataset-informed simulator, not a simulator learned from the data.

4.2 Kaggle Autonomous Navigation Control Dataset

The notebook downloads version 2 of the Autonomous Navigation Control Dataset using kagglehub, removes duplicate rows and prints the shape of the result: 1,000 records and 16 columns. The columns describe speed, steering angle, distance to obstacle, traffic-light state, pedestrian presence, weather, road type, lane position, camera and lidar similarity scores, distance to intersection, traffic-sign detection, steering and acceleration commands, and command labels. Of the 16 columns, only Distance_to_Obstacle_m and Weather_Condition are used by the simulator. Tables 4.1 and 4.2 summarise the same 1,000-row file; they were obtained during the dataset inspection of the earlier notebook version of this project, because the final notebook prints only the table shape and the first five records.

Table 4.1: Summary statistics of the numeric columns of the dataset

Numeric column	Mean	Std. dev.	Minimum	Maximum
Speed_kmph	59.760	35.582	0	120
Steering_Angle_deg	−0.622	26.094	−44.583	44.947
Distance_to_Obstacle_m	50.454	28.619	0.656	99.836
Pedestrian_Presence	0.495	0.500	0	1
Lane_Position	1.703	1.000	0.007	3.497
Camera_Input_Similarity	0.501	0.288	0.001	1.000
Lidar_Point_Cloud_Similarity	0.513	0.289	0.003	0.998
Distance_to_Intersection_m	99.644	58.602	0.271	199.948
Traffic_Sign_Detected	0.514	0.500	0	1
Steering_Command	−0.021	0.804	−1	1
Acceleration_Command	0.014	0.811	−1	1

Table 4.2: Categorical columns of the dataset

Categorical column	Observed categories / cardinality
Traffic_Light_State	3 categories
Weather_Condition	4 categories: Snowy, Foggy, Rainy, Sunny
Road_Type	3 categories
Steering_Command_Label	3 categories
Acceleration_Command_Label	3 categories

The dataset statistics are descriptive. They characterise the downloaded table and do not imply that the values are representative of real-world traffic distributions. The notebook does not conduct a statistical validation of the data-generating process.

4.3 Suitability of the Dataset for Reinforcement Learning

Table 4.3: Reinforcement-learning components identified in the dataset

RL component	Relevant field found	Interpretation
Position	Lane_Position	A lane-position feature is available, but not a full grid or pose trajectory.
Speed / acceleration	Speed_kmph, Acceleration_Command	Descriptive driving and control fields.
Heading	Steering_Angle_deg	Steering angle is available.
Sensors / obstacles	Distance_to_Obstacle_m, camera/lidar similarity	Sensor-related measurements are available.
Control / action	Steering_Command, Acceleration_Command	Control command fields are present.
Goal / reward / next state / terminal / episode	None identified	The dataset is not a complete RL transition log.

The absence of sequential identifiers means that consecutive rows cannot safely be interpreted as successive observations from one vehicle trajectory. In addition, the file does not state the reward received after an action or the resulting next state. These gaps prevent direct online interaction or a reliable reconstruction of transitions. The notebook therefore retains its own environment dynamics and uses only two dataset-derived simulator parameters.

4.4 Derivation of Obstacle Density

The obstacle-distance threshold is set to 15.0 m as an explicit project assumption. The raw density is calculated as the fraction of dataset records for which Distance_to_Obstacle_m is below this threshold. The notebook records a value of 0.155. This value is clipped to the interval [0.05, 0.25] to prevent the generated grid from becoming either too sparse or excessively obstructed. In the recorded run the value remains 0.155 after clipping.

The environment uses a random map generator with seed 123. It samples obstacle positions according to the selected density, clears the start and goal cells, and repeats the generation until breadth-first search verifies a route. The resulting map contains seven obstacle cells out of 64, giving a realised density of $7/64 \approx 0.109$. The realised density differs from the requested probability because each cell is drawn independently (so a 64-cell grid fluctuates around the requested value), the start and goal cells are always cleared, and only layouts that pass the breadth-first-search check are accepted.

4.5 Derivation of Weather-Dependent Slip

Table 4.4: Weather categories and assigned slip probabilities

Weather category	Rows	Share	Assigned slip probability
Snowy	268	0.268	0.15
Foggy	252	0.252	0.12
Rainy	248	0.248	0.10
Sunny	232	0.232	0.05
Overall dataset mean	1,000	1.000	0.107

The notebook defines the mapping in SLIP_BY_WEATHER. Each episode samples one slip value from the array constructed from all dataset rows. The selected value remains fixed for the duration of that episode. This design creates episode-level variation in action reliability while preserving a reproducible distribution across evaluation episodes. The numerical mapping is an assumption: the dataset contains weather labels but does not contain measured action-slip events.

4.6 Simulated Navigation Environment

Table 4.5: Configuration of the dataset-informed navigation environment

Item	Setting
Grid size	8 × 8 cells
Start / goal	(0, 0) / (7, 7)
Obstacle cells	(0,1), (3,1), (5,0), (5,6), (5,7), (6,2), (6,4)
Requested obstacle density	0.155, derived from dataset and clipped to [0.05, 0.25]
Map seed	123
Action set	0 up, 1 down, 2 left, 3 right
Slip probability	Episode-specific value sampled from dataset-derived weather mapping; mean 0.107
Observation	8 numeric values in Box(−1, 1), shape (8,)
Maximum episode length	100 steps

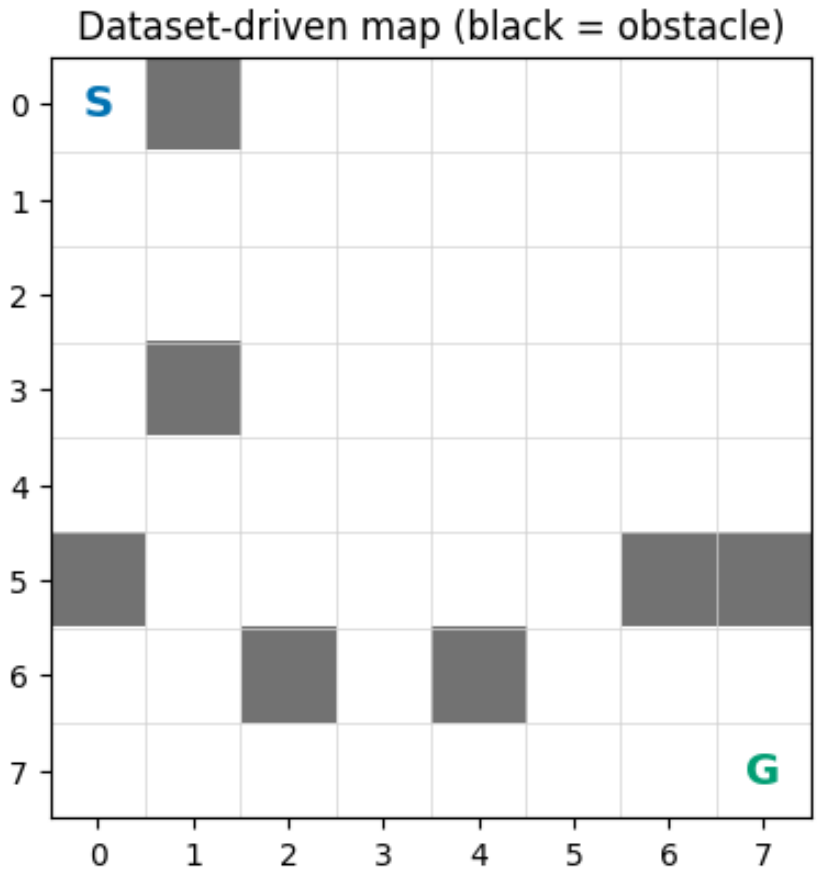


Figure 4.1: Dataset-informed simulated 8 × 8 navigation map generated by the notebook

4.7 Environment Interface and Experience Generation

The NavigationEnv class follows the Gymnasium interface [8]. reset() returns the initial observation and step(action) returns the next observation, reward, terminated flag, truncated flag and an information dictionary. The terminated flag records a true terminal event such as goal or collision; truncated records the 100-step time limit. This distinction is important because value targets should not treat a time-limit truncation as an absorbing terminal state.

All training transitions are generated online. Q-Learning and SARSA update their Q-table after each step. DQN stores transitions in a replay buffer of capacity 20,000 and samples mini-batches of 64 after the first 500 environment steps. PPO collects rollouts of 500 steps and performs ten epochs of mini-

batch optimisation before discarding the rollout. All algorithms receive the same number of environment steps, but not the same number of gradient updates or computational operations.

4.8 Data Quality, Preprocessing and Derived Parameters

The dataset loading stage uses kagglehub to retrieve the named Kaggle dataset when no local DATA_FILE is supplied. The notebook reads the CSV into a pandas DataFrame, removes duplicate rows and resets the row index. It identifies the required fields by searching for column names containing the substrings distance_to_obstacle and weather. This makes the implementation less dependent on exact capitalisation while still requiring the relevant columns to exist.

The recorded dataset shape after duplicate removal is 1,000 rows by 16 columns. The selected obstacle-distance column is continuous. The notebook converts it into a binary indicator by comparing each record with the assumed 15.0 m threshold. The mean of this indicator is the raw close-obstacle fraction, 0.155. The value is then clipped to [0.05, 0.25] before it is passed to the grid generator. The recorded value is already inside this interval, so clipping does not alter it.

Weather categories are normalised by converting their string representation to lower case and removing surrounding whitespace. A dictionary maps supported category labels to the assigned slip probability. The weather table counts the number of rows in each category, computes its share of the dataset, and reports the assigned slip value. Each row is mapped into the SLIP_VALUES array, which is sampled at environment reset. The recorded weather distribution is 26.8% Snowy, 25.2% Foggy, 24.8% Rainy and 23.2% Sunny.

No imputation or label encoding is required for these two derived parameters. The rest of the dataset remains available for descriptive inspection but is not used by the simulator dynamics. The notebook does not train a predictive model on the data and does not infer rewards. This restricted use keeps the relationship between the observed columns and simulator configuration transparent.

Chapter 5

Methodology

5.1 Overall Workflow

The workflow begins by loading the Kaggle dataset and identifying the obstacle-distance and weather columns. It then calculates obstacle density and constructs the episode-level slip distribution. A solvable grid is generated with a fixed map seed. The four learning algorithms are trained independently for each of three random seeds. Each resulting agent is evaluated on fixed seeds, metrics are aggregated and plots are produced.

The order of these stages preserves the separation between dataset-derived information and simulator assumptions. The 15 m obstacle threshold and weather-to-slip mapping are explicitly chosen by the project. The dataset supplies the measured obstacle-distance values and weather labels; it does not supply the simulator reward function or a complete transition model.

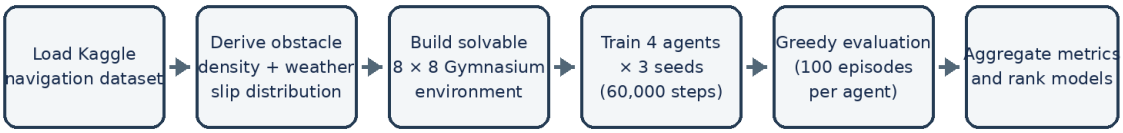


Figure 5.1: Workflow of the dataset-informed comparative study

5.2 Environment Initialisation and Map Validation

The NavigationEnv constructor stores grid size, episode limit, slip values, optional slip override, start and goal coordinates, and the generated obstacle map. The _make_map method uses NumPy's random generator with map seed 123. It samples a Boolean obstacle array and clears the start and goal positions. The _solvable method performs breadth-first search over the four movement directions. A map is accepted only when the goal can be reached.

Breadth-first search explores states in non-decreasing path length. In an unweighted grid, the first discovered path to the goal is therefore a shortest path. In this project it is used as a feasibility test, not as the navigation policy used by the RL agents. The fixed map isolates algorithm comparisons from map-to-map variation, but limits generalisation claims.

5.3 State and Action Representation

The tabular agents use the discretize function to recover integer row and column coordinates from the first two observation values. The flattened state is

$$s = 8r + c, \quad r, c \in \{0, \ldots, 7\} \tag{5.1}$$

so that $s \in \{0, \ldots, 63\}$. The 64 states and four actions form a Q-table with 256 entries. The neural agents receive the original eight-element observation vector. All four algorithms select from the same four movement actions.

5.4 Reward Calculation

The step method first samples whether the intended action slips, using the current episode's weather-derived slip probability. It then computes the executed destination and applies the reward rules. The info dictionary records is_success and collision flags. These flags are used directly to compute outcome metrics; the report does not infer success or collision from reward magnitude alone.

5.5 Agent Initialisation and Randomness

The tabular Q-table is initialised to zero. The DQN online network and target network begin with identical parameters. PPO is initialised through Stable-Baselines3 using MlpPolicy. The seed-setting function seeds Python, NumPy and PyTorch for the deep methods; the tabular training function uses a NumPy generator initialised with the run seed. The environment map seed remains fixed independently of the training seed.

5.6 Exploration and Exploitation

Q-Learning, SARSA and DQN use ε-greedy action selection. The exploration rate decreases linearly from 1.0 to 0.05 over 60% of the training budget and remains at 0.05 thereafter. With T = 60,000 and f = 0.6, the schedule is

$$\varepsilon(t) = \varepsilon_{start} + \min(1, t / (fT)) \cdot (\varepsilon_{end} - \varepsilon_{start}) \tag{5.2}$$

where $\varepsilon_{start} = 1.0$ and $\varepsilon_{end} = 0.05$. It reaches 0.05 at step 36,000. PPO samples actions from its stochastic policy during training and includes an entropy coefficient of 0.01.

During training, two sources of randomness affect the executed action: the exploration policy and environment slip. During greedy evaluation, ε-greedy exploration is removed and PPO uses deterministic prediction, but episode-level slip remains active. Thus, evaluation measures a deterministic decision rule operating under the simulator's stochastic transitions.

5.7 Q-Learning

Q-Learning is an off-policy temporal-difference algorithm. After observing transition (s,a,r,s′), its update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, [\, r + \gamma \max_{a'} Q(s', a') - Q(s, a) \,] \tag{5.3}$$

The learning rate is $\alpha = 0.1$ and the discount factor is $\gamma = 0.99$. For true terminal transitions, the target is the immediate reward. For truncation at the time limit, the next-state value remains in the target.

The maximum over next-state actions evaluates the greedy continuation even when the behaviour policy selected a different exploratory action. This is the defining off-policy distinction. The method is simple and computationally inexpensive, but each state-action estimate is stored separately and cannot share statistical strength across neighbouring observations.

5.8 SARSA

SARSA is an on-policy temporal-difference method. Its update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, [\, r + \gamma \, Q(s', a') - Q(s, a) \,] \tag{5.4}$$

where a′ is the action selected by the current ε-greedy policy in the next state. The update therefore evaluates the policy that is actually followed, including its remaining exploratory behaviour. In the notebook, Q-Learning and SARSA share the train_tabular function; only the target expression changes.

The on-policy target can reflect the consequences of exploratory actions and may favour routes that are more tolerant of action uncertainty. However, the current experiment does not isolate this explanation experimentally. The observed SARSA result is a measured outcome, while any claim about why it outperformed the other methods remains an interpretation.

5.9 Deep Q-Network

DQN approximates the action-value function using a neural network with an 8–64–64–4 architecture and ReLU hidden activations. The replay buffer has capacity 20,000 and stores observation, action, reward, next observation and true-terminal indicator. Once 500 transitions have been collected, the algorithm samples batches of 64 and performs one optimiser update per environment step. The target network is synchronised every 250 steps.

The target of the network is

$$y = r + \gamma\,(1 - d)\,\max_{a'} Q_{target}(s', a') \tag{5.5}$$

where d indicates true termination. The loss is SmoothL1 (Huber) loss and the optimiser is Adam with learning rate 10^{-3}. Gradient norm is clipped at 10. Experience replay reduces correlation between consecutive transitions and reuses stored experience; the target network stabilises the bootstrap target [4,9,10].

5.10 Proximal Policy Optimization

PPO is an actor-critic algorithm. The actor produces action probabilities and the critic estimates state value. The implementation uses Stable-Baselines3 PPO with MlpPolicy, two hidden layers of 64 units, n_steps = 500, batch_size = 100, n_epochs = 10, learning_rate = 3×10^{-4}, gamma = 0.99, gae_lambda = 0.95, clip_range = 0.2 and ent_coef = 0.01 [5,11,12].

PPO forms a probability ratio between the updated and previous policy and optimises a clipped surrogate objective. Clipping limits the incentive for a single update to move the policy too far from the policy that generated the rollout. Generalised advantage estimation combines temporal-difference residuals over multiple steps and balances bias and variance. PPO uses each rollout for several optimisation epochs and then discards it.

With the probability ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta old}(a_t \mid s_t)$ and an advantage estimate $\hat{A}_t$, the clipped surrogate objective is

$$L^{CLIP}(\theta) = E_t[\ \min(\ \rho_t(\theta)\,\hat{A}_t,\ \ \mathrm{clip}(\rho_t(\theta),\, 1 - \varepsilon,\, 1 + \varepsilon)\,\hat{A}_t\ )\ ] \tag{5.6}$$

where $\varepsilon = 0.2$ is the clip range used in the notebook [5].

5.11 Training and Evaluation Workflow

Each algorithm is trained for 60,000 environment steps for each of seeds 0, 1 and 2. The notebook records completed episode reward, success, collision, length and the environment step at which each episode ends. Training wall-clock time is measured. Each trained agent is then evaluated for 100 episodes using reset seeds 10,000 through 10,099. The same evaluation seed range is used for all agents.

5.12 Model Comparison Procedure

The environment, fixed map, reward function, total environment-step budget, number of training seeds and evaluation episodes are shared. Results are reported as mean and population standard deviation over the three seeds. The ranking sorts by mean success rate, then mean reward, then collision rate. Because

the algorithms differ in representation, exploration and experience reuse, equal environment steps do not imply equal computational budgets.

The common 60,000-step budget controls the amount of environment interaction but does not equalise computation. DQN performs many mini-batch updates after warm-up, PPO performs repeated epoch updates per rollout, and the tabular methods perform inexpensive table updates. The training-time results should therefore be interpreted alongside this difference in experience reuse.

5.13 Algorithmic Update Sequence and Experience Reuse

The tabular learning loop maintains the current state and action, executes the action, observes the reward and next state, and applies a one-step temporal-difference update. Q-Learning constructs its target using the maximum action value in the successor state. SARSA first selects the next action using the current ε-greedy policy and uses the value of that selected action in its target. The two methods therefore share most of the training procedure while differing in the target definition.

DQN follows a separate sequence. The transition is appended to the circular replay buffer. Once the learning-start threshold has been reached, a random mini-batch is sampled and converted to tensors on the selected device. The online network predicts Q-values, the target network constructs the bootstrap target, and the SmoothL1 loss is minimised using Adam. The target network is periodically synchronised with the online network. The replay buffer allows transitions to be reused across several updates.

PPO does not use a replay buffer. It gathers a fixed-length rollout from the current policy, estimates returns and advantages, and optimises the clipped policy objective over multiple epochs. Once optimisation is complete, the rollout is discarded. This makes PPO on-policy in the sense that the data are associated with the policy that generated them. Its use of multiple epochs means that the same rollout contributes to several gradient updates before being discarded.

Chapter 6

Experimental Setup

6.1 Software and Execution Platform

Table 6.1: Software environment and execution platform

Item	Recorded detail
Programming language	Python; exact version not printed
Notebook execution	Linux environment; the notebook output shows the kagglehub cache path /root/.cache/kagglehub, and the hosting platform (for example Google Colab) is not declared
PyTorch	2.11.0+cu130
Gymnasium	1.4.0
Stable-Baselines3	Installed at run time; exact version not printed
NumPy, pandas, Matplotlib	Used; exact versions not printed
Hardware device	CPU used for DQN and PPO; GPU use disabled by USE_GPU_FOR_DEEP_RL = False
Dataset access	kagglehub download of freshersstaff/autonomous-navigation-control-dataset

The installed PyTorch build (2.11.0+cu130) is a CUDA build, but the notebook sets USE_GPU_FOR_DEEP_RL = False and prints device: cpu, so the deep algorithms execute on the CPU. The precise CPU model and RAM are not reported. These omissions matter for reproducibility of wall-clock timing, particularly for neural methods.

6.2 Environment and Shared Training Settings

Table 6.2: Settings shared by all algorithms

Setting	Value
QUICK_MODE	False
Training steps per run	60,000
Training seeds	0, 1, 2
Evaluation episodes per trained agent	100
Grid size	8 × 8
Maximum episode steps	100
Map seed	123
Obstacle threshold assumption	15.0 m
Obstacle density	0.155
Average weather-mapped slip	0.107
Discount factor	0.99

6.3 Algorithm Hyperparameters

Table 6.3: Hyperparameters of the four algorithms

Parameter	Q-Learning / SARSA	DQN	PPO
Learning rate	$\alpha = 0.1$	1×10^{-3}	3×10^{-4}
Discount factor	0.99	0.99	0.99
Exploration	ε-greedy, 1.0→0.05	ε-greedy, 1.0→0.05	Stochastic policy; entropy 0.01
Network	None	8–64–64–4	MlpPolicy, [64,64]
Replay buffer	None	20,000	None
Batch size	Not applicable	64	100
Learning start	Immediate	500 steps	Rollout 500
Target update	Not applicable	Every 250 steps	Not applicable
Epochs / updates	One table update per transition	One gradient update per step after warm-up	10 epochs per rollout
Gradient clipping	Not applicable	Norm 10	Library/default implementation settings unless explicitly set

6.4 Repeated Runs and Reproducibility

The notebook executes four algorithms for three seeds, giving 12 full training runs. The map seed remains fixed at 123, while the agent seeds vary across 0, 1 and 2. This design repeats learning under different random initialisation and stochastic sequences while keeping the map constant. The evaluation uses a fixed sequence of 100 reset seeds, providing a common evaluation protocol.

6.5 Evaluation Protocol

The trained Q-Learning and SARSA agents select the maximum-valued table action. DQN selects the action with the highest network Q-value. PPO uses deterministic prediction. Environment slip remains active and is sampled from the dataset-derived distribution at each reset. Evaluation results are not used to alter training hyperparameters.

6.6 Training Time Measurement

Training time is recorded using time.time() around each training run. The values therefore represent elapsed wall-clock time on the execution environment, not a hardware-independent measure of algorithmic complexity. The times are useful for comparison within the recorded run but should not be treated as universal benchmarks.

6.7 Reproducibility Controls and Recorded Uncertainties

The notebook fixes the map seed at 123 and uses a separate list of agent seeds, [0, 1, 2]. The seed-setting function initialises Python, NumPy and PyTorch random generators. The tabular agent uses a NumPy generator seeded with the run seed. These controls make the experiment repeatable within the same software and hardware environment, subject to the reproducibility properties of the underlying libraries.

The evaluation function uses a fixed seed base of 10,000 and increments it for each of the 100 evaluation episodes. This gives every trained policy the same sequence of environment reset seeds. The environment still samples a weather-derived slip value for each episode. The evaluation protocol is greedy with respect to the learned policy, but the environment remains stochastic.

Some implementation details are not recorded. The notebook prints PyTorch and Gymnasium versions but does not print the exact Python, NumPy, pandas, Matplotlib or Stable-Baselines3 versions. It also does not record the CPU model or available RAM. These omissions do not change the saved metrics, but they limit the ability to reproduce wall-clock timings exactly on another machine.

The saved notebook outputs document the executed full experiment. The present report uses those outputs and does not rerun training. This avoids accidentally substituting a new random run for the results that are being documented. Any future rerun should preserve the source notebook, package versions, data version, seeds and execution settings together.

Chapter 7

Evaluation Metrics

The evaluation uses outcome, efficiency, learning-progress and computational metrics. The metrics are calculated from episode logs and greedy evaluation episodes, and the report keeps training statistics separate from final-policy evaluation statistics.

Table 7.1: Evaluation metrics used in this study

Metric	Definition	Interpretation
Average episode reward	Mean cumulative reward over evaluation episodes	Combines goal reward and movement/collision penalties.
Success rate	Successful episodes / total evaluation episodes	Primary measure of goal-reaching reliability.
Average steps to goal	Mean episode length among successful episodes	Measures route efficiency conditional on success.
Collision rate	Collision episodes / total evaluation episodes	Measures unsafe obstacle contact in the simulator.
Timeout rate	Timeout episodes / total evaluation episodes	Measures failure to terminate by goal or collision before the limit.
Convergence step	First logged step where 50-episode moving success average reaches 0.8	Descriptive learning-progress indicator.
Stability (reward std)	Standard deviation of episode rewards recorded after 80% of training budget	Describes late-training reward variability.
Training time	Wall-clock seconds for one complete run	Computational cost on the recorded platform.

7.1 Success Rate

Success rate is computed as the mean of the binary is_success flag over evaluation episodes. It is the primary ranking metric because the task is to reach the goal. A rate of 0.827 corresponds to approximately 82.7 successful episodes per 100 evaluation episodes, averaged across the three independently trained agents.

7.2 Reward and Route Length

Average reward captures the combined effects of reaching the goal, movement costs, boundary actions and collisions. Average steps to goal is calculated only for successful episodes. It should therefore be interpreted together with success rate: an agent may have a low average successful route length while failing many episodes.

7.3 Collision and Timeout Rates

Collision rate and timeout rate partition unsuccessful outcomes. Collision is read from the environment info flag. Timeout rate is calculated as one minus success rate minus collision rate. Floating-point rounding can produce values close to zero but displayed as −0.00; this is a numerical display artefact, not a negative probability.

7.4 Convergence and Stability

The convergence indicator uses a moving average of 50 completed-episode success flags and records the first end step at which this moving average reaches 0.8. It is a descriptive threshold, not a formal proof of convergence to an optimal policy. Stability is the standard deviation of episode rewards logged after the 80% training point. It measures late-training reward dispersion and is not identical to variability of greedy evaluation success.

7.5 Aggregation Across Seeds

The notebook reports the mean and population standard deviation across three seed-level results. Population standard deviation divides by N = 3. It is a descriptive summary of the observed runs, not a confidence interval. The small number of seeds makes strong statistical claims inappropriate.

The standard deviation across seeds describes how much the results vary between independently trained agents. It is not the same as the standard deviation across episodes within one trained agent. In the aggregate table, Q-Learning and PPO have large seed-level deviations because one or more runs are markedly worse than the others. DQN has a smaller success-rate deviation in the three recorded runs.

7.6 How to Read the Evaluation Metrics Together

Training convergence and final greedy evaluation answer different questions. The convergence indicator is based on a moving average of training episode success, during which exploration remains active. Evaluation uses greedy actions. A convergence step therefore does not guarantee a particular final evaluation success rate, and the two measures should not be substituted for each other.

Average reward combines the terminal reward and the costs incurred during an episode. A high success rate generally contributes positively through the +100 goal reward, but the final reward also depends on movement count, boundary actions and collision outcomes. The DQN result illustrates this distinction: it has a slightly lower mean success rate than SARSA but a higher mean reward and shorter successful routes. SARSA has the higher success frequency and lower collision fraction.

The success rate is the primary measure of whether the policy fulfils the navigation objective, but it should not be read in isolation. A policy may have a high average route length because it succeeds only after taking detours, or it may have a low average route length because it succeeds quickly in a subset of episodes while failing many others. For this reason, average steps to goal is reported only for successful episodes and is interpreted together with success rate.

Chapter 8

Results and Analysis

This chapter reports the saved outputs of the full notebook run with QUICK_MODE = False, TRAIN_STEPS = 60,000, seeds 0, 1 and 2, and 100 greedy evaluation episodes per trained agent. The figures are extracted from the notebook output and the tables reproduce its recorded per-seed and aggregate values. No additional training run was performed while preparing this report.

8.1 Training Behaviour

The notebook completed all 12 runs. Q-Learning completed 4,306, 3,932 and 3,977 training episodes for seeds 0, 1 and 2. SARSA completed 3,687, 3,750 and 3,669 episodes. DQN completed 4,424, 4,490 and 4,440 episodes. PPO completed 28,841, 26,242 and 4,230 episodes. The difference in episode counts reflects different episode lengths and learning behaviour under the same environment-step budget.

Table 8.1: Number of training episodes completed in 60,000 environment steps

Algorithm	Seed 0 episodes	Seed 1 episodes	Seed 2 episodes
Q-Learning	4,306	3,932	3,977
SARSA	3,687	3,750	3,669
DQN	4,424	4,490	4,440
PPO	28,841	26,242	4,230

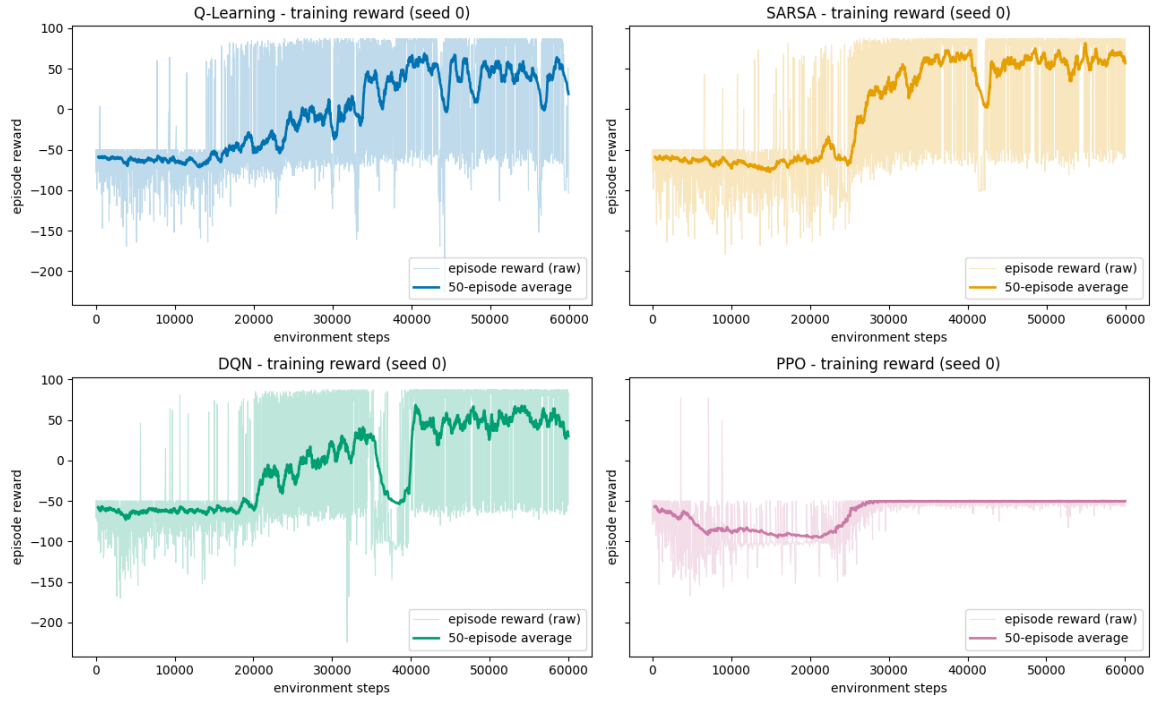


Figure 8.1: Training reward of seed 0 for each algorithm, including raw episode reward and moving average

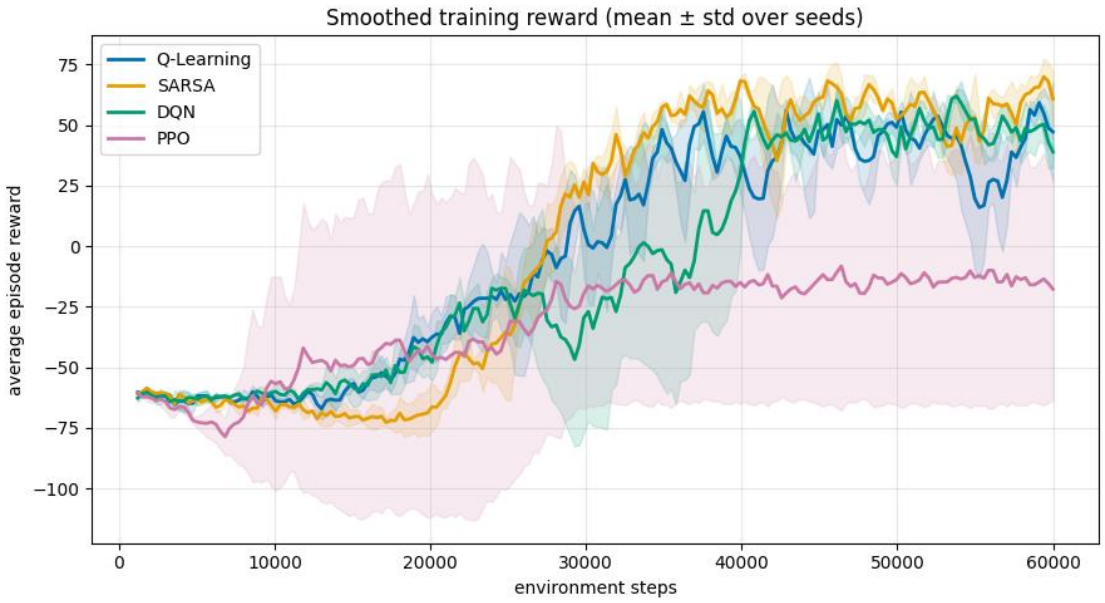


Figure 8.2: Smoothed training reward, mean ± standard deviation over three seeds



Figure 8.3: Smoothed training success rate, mean ± standard deviation over three seeds

Training curves include exploratory behaviour and therefore need not match the greedy evaluation results. The curves also aggregate different episode counts at common environment-step locations. They provide a view of training progress but do not replace the final evaluation table.

8.2 Per-Seed Greedy Evaluation

Table 8.2: Greedy evaluation results for each trained agent

Algorithm	Seed	Mean reward	Success	Steps to goal	Collision	Timeout
Q-Learning	0	−93.35	0.02	58.000	0.16	0.82
Q-Learning	1	63.32	0.85	15.835	0.15	0.00
Q-Learning	2	65.93	0.87	16.000	0.13	0.00
SARSA	0	64.54	0.86	15.860	0.14	0.00
SARSA	1	63.42	0.85	15.718	0.15	0.00
SARSA	2	32.32	0.77	34.584	0.14	0.09
DQN	0	60.59	0.83	15.602	0.17	0.00
DQN	1	58.86	0.82	15.463	0.18	0.00
DQN	2	53.61	0.78	15.551	0.22	0.00
PPO	0	−50.37	0.00	n/a	1.00	0.00
PPO	1	−50.37	0.00	n/a	1.00	0.00
PPO	2	52.15	0.77	15.494	0.23	0.00

The seed-level results show substantial variation for Q-Learning and PPO. Q-Learning seed 0 reaches the goal in only 2% of evaluation episodes and has a timeout rate of 0.82. PPO seeds 0 and 1 collide in all 100 episodes, while seed 2 reaches a success rate of 0.77. SARSA has a narrower range of success rates, from 0.77 to 0.86, and DQN ranges from 0.78 to 0.83.

8.3 Aggregate Evaluation Results

Table 8.3: Evaluation metrics, mean ± population standard deviation over seeds

Algorithm	Average reward	Success rate	Steps to goal	Collision rate
Q-Learning	12.0 ± 74.5	0.580 ± 0.396	29.9 ± 19.8	0.147 ± 0.012
SARSA	53.4 ± 14.9	0.827 ± 0.040	22.1 ± 8.9	0.143 ± 0.005
DQN	57.7 ± 3.0	0.810 ± 0.022	15.5 ± 0.1	0.190 ± 0.022
PPO	−16.2 ± 48.3	0.257 ± 0.363	15.5 ± 0.0	0.743 ± 0.363

SARSA obtains the highest mean success rate at 0.827. DQN is second at 0.810 and has the highest mean reward at 57.7 and the shortest successful-route length at 15.5 steps. SARSA's average route length is higher because seed 2 contains successful episodes with a mean length of 34.584. Q-Learning and PPO have high seed variation driven by one or more poor runs.

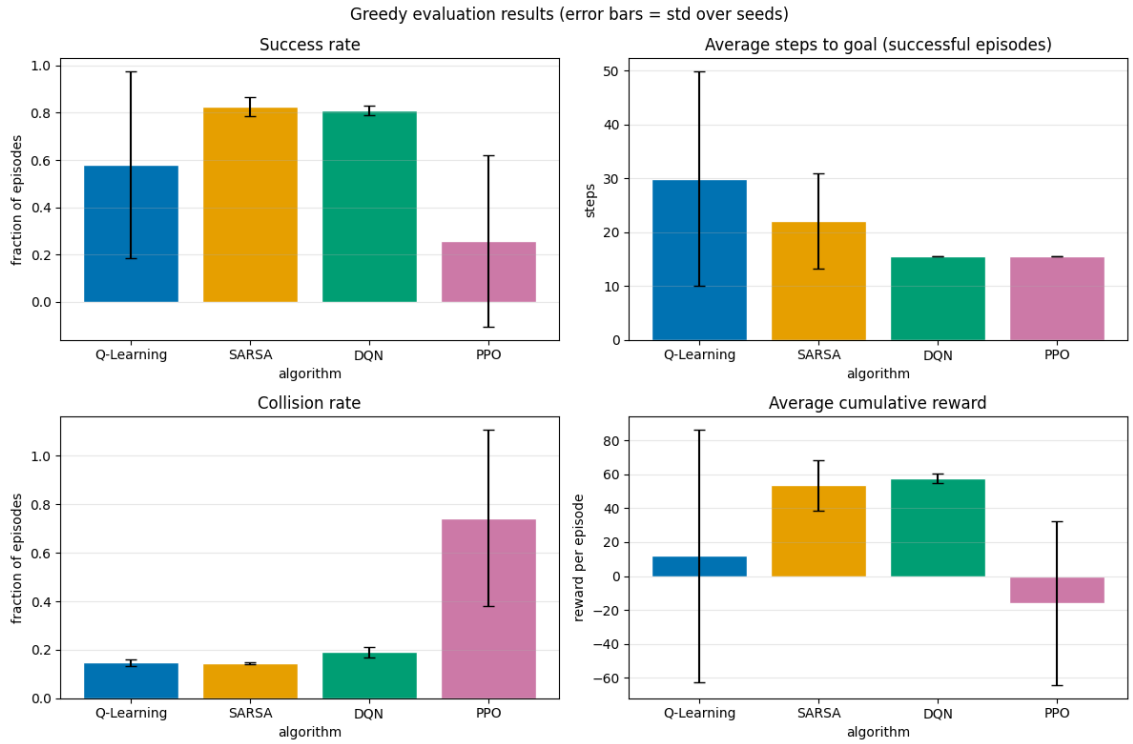


Figure 8.4: Greedy evaluation metrics for the four algorithms; error bars show standard deviation across seeds

8.4 Median Results and Ranking

Table 8.4: Median evaluation results over the three seeds (derived from Table 8.2)

Algorithm	Median success rate	Median average reward	Mean success rate (for comparison)
Q-Learning	0.85	63.32	0.580
SARSA	0.85	63.42	0.827
DQN	0.82	58.86	0.810
PPO	0.00	−50.37	0.257

The median success rates are 0.85 for Q-Learning, 0.85 for SARSA, 0.82 for DQN and 0.00 for PPO. Median reward values are 63.32, 63.42, 58.86 and −50.37 respectively. These medians show that the average Q-Learning result is strongly affected by its poor seed 0, while PPO's two poor seeds dominate its median.

Table 8.5 orders the algorithms with the rule used in the notebook: mean success rate first, then mean reward, then collision rate. The training times in the last column are the means of the three runs.

Table 8.5: Algorithm ranking using mean success rate and tie-break metrics

Rank	Algorithm	Mean success	Mean reward	Collision rate	Mean successful steps	Training time (s)
1	SARSA	0.827	53.427	0.143	22.054	2.126
2	DQN	0.810	57.687	0.190	15.539	132.599
3	Q-Learning	0.580	11.967	0.147	29.945	2.087
4	PPO	0.257	−16.197	0.743	15.494	69.722

8.5 Convergence, Stability and Training Time

Table 8.6: Convergence, stability and training time

Algorithm	Convergence step (mean ± SD)	Seeds reaching threshold	Late reward SD	Training time (s)
Q-Learning	36,387 ± 2,295	3/3	64.1 ± 0.8	2.1 ± 0.1
SARSA	32,747 ± 1,370	3/3	55.7 ± 2.0	2.1 ± 0.4
DQN	39,347 ± 2,069	3/3	61.9 ± 1.1	132.6 ± 2.4
PPO	16,836 ± 0	1/3	18.8 ± 25.1	69.7 ± 0.4

The tabular methods are the least expensive to train, each requiring about two seconds per run. PPO requires about 69.7 seconds, while DQN requires about 132.6 seconds. DQN is the most computationally expensive in this run. The reported convergence step for PPO is available for only one of three seeds because the other two do not reach the 0.8 moving-success threshold.

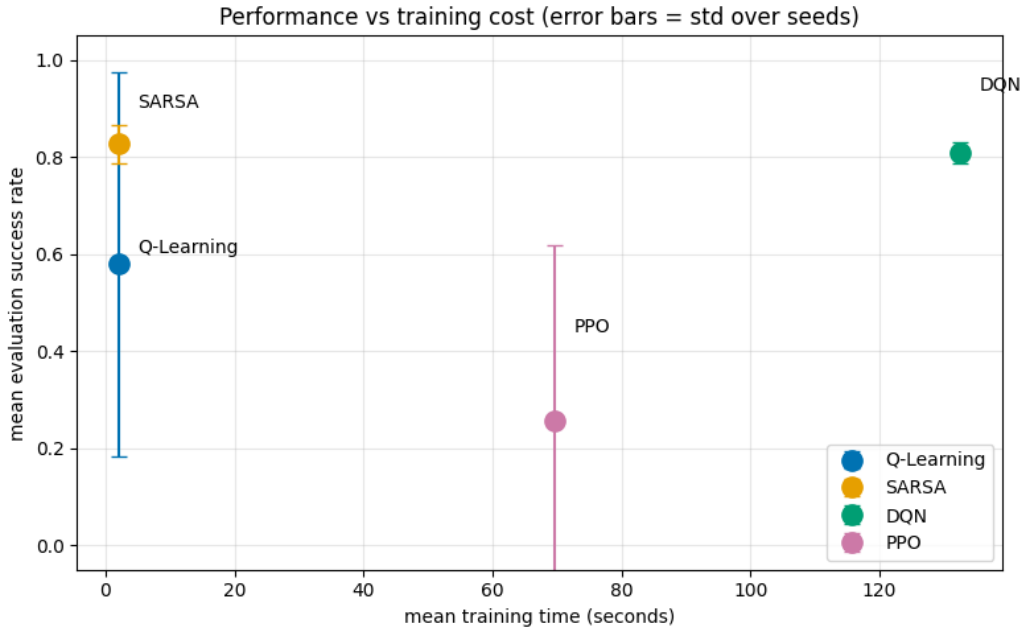


Figure 8.5: Mean evaluation success rate against mean training time; error bars show seed standard deviation

8.6 Best-Model Identification

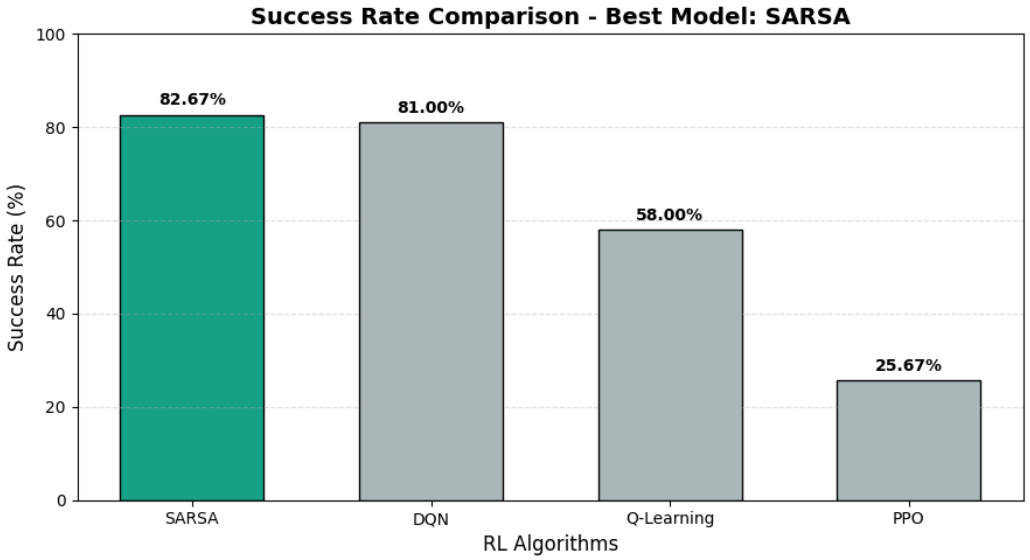


Figure 8.6: Success-rate comparison highlighting the highest mean result, SARSA

The notebook's ranking rule selects the algorithm with the highest mean success rate, with mean reward and collision rate used as tie-breakers. SARSA is ranked first with a mean success rate of 0.827 (82.7%). Its lead over DQN is 0.017. The seed standard deviations are 0.040 for SARSA and 0.022 for DQN. Since the lead is not clearly larger than seed-to-seed variation, the ranking should be treated as indicative rather than proof of a statistically reliable difference.

8.7 Summary of Experimental Findings

The main recorded findings are: SARSA has the highest mean success rate; DQN has the highest mean reward and shortest mean successful route; the tabular methods are substantially faster to train than the deep methods; Q-Learning and PPO show large sensitivity to seed in this experiment; and the ranking is not statistically conclusive with only three seeds.

8.8 Cross-Algorithm Analysis of the Recorded Results

The ranking therefore depends on the performance criterion. SARSA is the winner under mean success rate. DQN is strongest on mean reward and successful-route length. The tabular methods are fastest to train. A recommendation should state which criterion is being prioritised rather than presenting one algorithm as best in every respect.

DQN takes approximately 132.6 seconds per run, the longest of the four methods. Its mean successful-route length is 15.5 steps, close to the 14-step theoretical minimum, and its standard deviation in that metric is low. Its collision rate is higher than the tabular methods in this run. PPO takes approximately 69.7 seconds and exhibits two seed runs with zero success. Its single successful seed is insufficient to offset those failures in the mean.

The tabular methods have the lowest training times. Q-Learning averages approximately 2.1 seconds and SARSA approximately 2.1 seconds. These methods also have different outcome patterns: SARSA is comparatively consistent across the three seeds, whereas Q-Learning has one very poor run. The shared table representation and low computational overhead make the methods attractive for small discrete environments, but the results show that a tabular method can still be sensitive to the random training sequence.

The aggregate table places SARSA first by mean success rate, DQN second, Q-Learning third and PPO fourth. The difference between SARSA and DQN is 0.017. This difference is modest when compared with the seed standard deviations of 0.040 and 0.022. Consequently, the ordering is a transparent summary of the measured values rather than a statistically established separation between the top two algorithms.

Chapter 9

Sample Predictions and Navigation Behaviour

9.1 Greedy Paths of the Seed-0 Agents

The notebook plots one greedy trajectory for each trained agent from seed 0 with slip noise disabled solely for visual inspection. This illustration is different from the main evaluation, where episode-level slip remains active. Turning off slip for the plot makes it easier to inspect the intended policy path and does not change the evaluation values reported in Chapter 8.

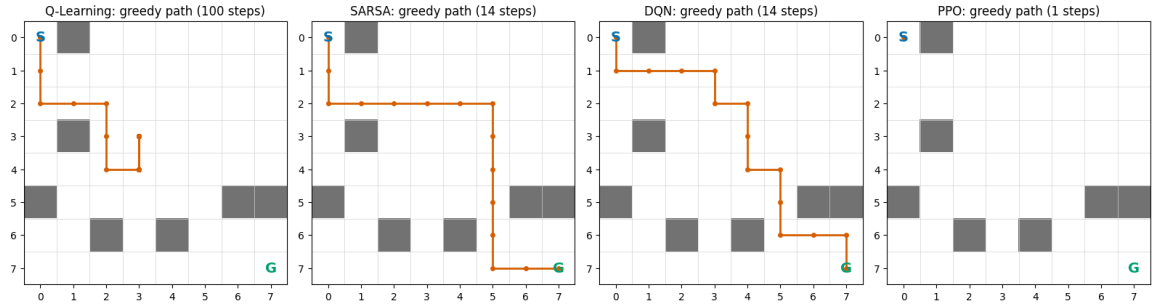


Figure 9.1: Greedy paths of the seed-0 agents with slip noise disabled for illustration

The shortest possible path has 14 actions. A successful route of this length indicates that the policy reaches the goal without detours in the deterministic illustration. The path plot should be read as a single-policy example rather than a statistical summary of the 100-episode evaluation.

Table 9.1 reads the four paths from Figure 9.1. SARSA and DQN reach the goal in 14 steps, which equals the minimum route length. The seed-0 Q-Learning agent does not reach the goal even with slip disabled: its path turns south in the upper-left part of the map, loops and is cut off at the 100-step limit. The seed-0 PPO agent chooses the move to the right from the start cell, which leads into the obstacle at (0, 1), so its episode ends in a collision after one step.

Table 9.1: Routes of the seed-0 greedy agents without slip noise (read from Figure 9.1)

Algorithm	Steps	Outcome	Route read from the figure
Q-Learning	100	Timeout; goal not reached	(0,0) down to (2,0), east to (2,2), south through (3,2) and (4,2), then loops around (4,3) and (3,3) until the step limit
SARSA	14	Goal reached	(0,0) down to (2,0), east along row 2 to (2,5), south along column 5 to (7,5), east to (7,7)
DQN	14	Goal reached	(0,0) down to (1,0), east to (1,3), then a staircase through (2,3), (2,4), (3,4), (4,4), (4,5), (5,5), (6,5), east to (6,7), down to (7,7)
PPO	1	Collision	Moves right from (0,0) into the obstacle at (0,1); the episode ends immediately

9.2 Interpretation of Navigation Outcomes

The no-slip paths in Figure 9.1 help to explain part of the stochastic evaluation results. The seed-0 Q-Learning policy does not reach the goal even without slip: its greedy route loops and runs to the 100-step limit, which is consistent with its evaluation success rate of 0.02 and timeout fraction of 0.82. The seed-0 PPO policy selects a move into the obstacle at (0, 1) from the start cell, which is consistent with its collision fraction of 1.00. For these two agents the poor evaluation result is therefore a property of the learned greedy policy and not only of the action slip. The notebook does not analyse why these two policies failed.

The three DQN seeds have comparatively close success rates and successful route lengths near 15.5 steps. This suggests relatively consistent behaviour across the recorded seeds, although the collision rate is higher than that of the tabular methods. SARSA has the best mean success rate but a larger mean successful-route length because of the slower successful episodes in seed 2.

9.3 Outcome Fractions

Table 9.2: Outcome fractions for the trained agents

Algorithm	Seed 0 success / collision / timeout	Seed 1 success / collision / timeout	Seed 2 success / collision / timeout
Q-Learning	0.02 / 0.16 / 0.82	0.85 / 0.15 / 0.00	0.87 / 0.13 / 0.00
SARSA	0.86 / 0.14 / 0.00	0.85 / 0.15 / 0.00	0.77 / 0.14 / 0.09
DQN	0.83 / 0.17 / 0.00	0.82 / 0.18 / 0.00	0.78 / 0.22 / 0.00
PPO	0.00 / 1.00 / 0.00	0.00 / 1.00 / 0.00	0.77 / 0.23 / 0.00

The outcome fractions clarify why mean reward and success rate can tell different stories. PPO seed 0 and seed 1 collide in every evaluation episode, resulting in a reward of −50.37 and a success rate of zero. Q-Learning seed 0 has a high timeout fraction, which reduces its average reward substantially. SARSA seed 2 has a timeout fraction of 0.09. These observations are directly derived from the recorded evaluation table.

9.4 Interpretation of a Single Greedy Trajectory

A greedy trajectory is a deterministic sequence of actions selected by a trained policy when the environment is configured without action slip for the purpose of plotting. It helps reveal the route implied by the learned values or policy, but it is not an estimate of expected performance under the stochastic evaluation environment. The path plot is therefore an explanatory visualisation rather than a replacement for the 100-episode evaluation.

The shortest possible path from the upper-left to the lower-right corner has 14 moves. A plotted route that uses more than 14 moves contains a detour. A route with 14 moves is geometrically shortest, although it may still be vulnerable to action slip when evaluated under stochastic transitions. The figure allows a visual check of whether the learned policy avoids the obstacle cells and reaches the goal in the deterministic illustration: SARSA and DQN do, whereas the Q-Learning and PPO seed-0 policies do not.

This distinction prevents over-interpretation of the path visualisation. The plotted route can explain what the greedy policy attempts to do in a clean illustration, while success rate and collision rate quantify what happens when that policy interacts with the stochastic simulator.

The main evaluation keeps the sampled episode slip active. A single action deviation can move the agent away from the intended path, and the resulting state may require a recovery action. The notebook does not explicitly calculate the probability of recovering from each deviation or compare path risk across all possible trajectories. Such analysis would require a larger evaluation of transition outcomes or an explicit dynamic-programming analysis of the fixed map.

Chapter 10

Discussion

10.1 Findings in Relation to the Objectives

The notebook implements and evaluates all four algorithms under a common environment-step budget. It also uses the dataset to derive two simulator properties, which is a substantive change from a simulator that merely inspects the data. The primary comparison objective is addressed through per-seed and aggregate evaluation tables. The best mean success rate belongs to SARSA, but the small lead over DQN and the small number of seeds limit the strength of the conclusion.

10.2 Interpretation of the Best Result

SARSA achieves a mean success rate of 0.827, compared with 0.810 for DQN. The 1.7 percentage-point difference is modest relative to the variation across seeds. SARSA also has a lower mean collision rate (0.143) than DQN (0.190), but a longer mean successful-route length (22.1 versus 15.5). These metrics reveal a trade-off: SARSA is the highest-ranked method by success frequency, whereas DQN reaches the goal in fewer steps when it succeeds.

The on-policy nature of SARSA may help it account for exploratory behaviour in its learning target, but the present design does not isolate this factor from all other differences. The result should be described as an empirical observation from this configuration, not as proof that on-policy learning is generally better for navigation.

10.3 Dataset-Informed Simulation

The dataset contributes to the simulator in two explicit ways. The observed fraction of obstacle distances below 15 m sets the requested obstacle density, and the weather labels determine the distribution of episode-level slip probabilities. This creates a traceable connection between the tabular driving dataset and the simulation. Nevertheless, the 15 m threshold and the numerical weather mapping are assumptions chosen by the project. The data do not contain observed action slips or a measured causal relation between weather and control reliability.

10.4 Comparison of Learning Paradigms

The two tabular methods train rapidly because their updates modify a small table directly. DQN and PPO require neural-network computation and are substantially slower in the recorded CPU environment. DQN's replay buffer and target network introduce additional memory and update costs, while PPO performs repeated optimisation epochs over rollouts. The equal environment-step budget therefore compares sample efficiency under a shared interaction budget, not total computational effort.

DQN achieves the highest mean reward and the shortest mean successful route, but it does not have the highest success rate. PPO performs poorly in two seeds and successfully in one, indicating high sensitivity in the current setup. No single metric captures all dimensions of algorithm quality.

10.5 Reliability Across Seeds

Seed variability is central to interpreting the results. Q-Learning's mean success rate is reduced by a 0.02 run despite two runs at 0.85 and 0.87. PPO's mean is reduced by two runs with zero success.

SARSA and DQN have more consistent success rates. Reporting only a single seed would therefore have produced a misleading comparison.

10.6 Fairness and Threats to Validity

The comparison shares the map, reward function, environment-step budget, training seeds and evaluation episodes. However, the algorithms do not have identical observation processing: the tabular methods use a cell index, while DQN and PPO use the full observation vector. Their exploration strategies also differ. PPO samples from a stochastic policy rather than using ε-greedy selection. The methods also reuse experience differently and have different computational costs.

The environment is one fixed map, and the dataset-to-simulator transformations include assumptions. The experiment uses three seeds and no statistical hypothesis tests or confidence intervals. These conditions limit external validity and make small ranking differences uncertain. The results are still useful as a controlled case study of the four implementations on the same task.

10.7 Practical Implications

For this particular small grid task and implementation, SARSA is the best choice if the primary objective is the highest observed mean success rate and low collision rate. DQN may be preferable when successful-route length and average reward are emphasised, but it has a substantially higher training cost. The tabular algorithms are attractive when the state space is small and discrete. These observations should not be extrapolated directly to larger or continuous navigation tasks.

Chapter 11

Limitations

The following limitations arise from the data, environment design, implementation and experimental protocol.

1. Dataset completeness: the Kaggle file has no reward, next-state, terminal, episode or time-step fields. It cannot be used as a complete offline RL dataset or as an interactive environment.
2. Assumed obstacle threshold: the 15 m threshold is selected by the project. The resulting 0.155 density depends on this choice and could change under another threshold.
3. Assumed weather-to-slip mapping: numerical slip values are assigned to weather labels by the project. They are not calibrated from measured vehicle-control failures.
4. Discrete environment: the 8 × 8 grid and four movement actions omit continuous motion, vehicle dynamics, localisation, perception uncertainty and moving obstacles.
5. Fixed map: all runs use the same map seed. The results do not establish generalisation to different obstacle layouts.
6. Small number of seeds: only three training seeds are used. The observed variation is descriptive and no statistical significance testing is performed.
7. Unequal computational work: all algorithms receive the same environment-step budget, but their number and type of parameter updates differ.
8. Observation representation: tabular methods use a flattened cell index, while DQN and PPO use an eight-value vector. This is a representation difference within the comparison.
9. Hardware reporting: CPU model, RAM and some library versions are not printed, limiting exact reproduction of wall-clock training times.
10. Library dependence: PPO is implemented through Stable-Baselines3. The precise library version is not recorded, and default settings not explicitly set by the notebook may vary across versions.
11. No physical validation: the environment is simulated and does not establish safe or effective behaviour on a real robot or vehicle.

11.1 Consequences for the Conclusions

These limitations do not invalidate the recorded experiment, but they define the scope of its conclusions. SARSA has the highest mean success rate in the recorded configuration; this does not establish universal superiority. The data-informed simulator is a modelling construction with traceable inputs, not a calibrated digital twin. The evaluation supports comparative observations within the selected grid world only.

The report should also retain the distinction between training behaviour, deterministic greedy policy behaviour and stochastic environment outcomes. Training curves include exploration, the sample path plot disables slip for illustration, and the evaluation keeps episode-level slip active. Keeping these three views separate is necessary for a transparent account of what the agent learned and how it performed under the specified simulator.

11.2 Reproducibility and Reporting Recommendations

For stronger algorithm comparisons, the evaluation should be repeated over more training seeds and more map seeds. The same held-out map set should be used for all methods, and per-map as well as aggregate outcomes should be retained. Confidence intervals or non-parametric comparisons could then be used where their assumptions are appropriate. Such analysis would help determine whether the

observed difference between SARSA and DQN is robust or is a consequence of the particular random initialisations in the current experiment.

The data-to-simulator transformation should also be reported as a separate configuration layer. In particular, the 15 m threshold determines which distance observations count as close, and the weather-to-slip dictionary determines the uncertainty distribution used by the environment. Recording these values separately from the measured dataset statistics prevents a reader from confusing a modelling assumption with an empirical measurement. A future study could evaluate multiple thresholds and alternative mappings to quantify how sensitive the policy ranking is to these choices.

Future replications should preserve the complete notebook, the exact downloaded dataset version, the random seeds, the environment configuration and the installed package versions. The current notebook records the Kaggle dataset identifier and prints PyTorch and Gymnasium versions, but some dependencies and hardware details are not captured. A reproducibility record should therefore include the Python version, operating system, CPU model, RAM, GPU model where applicable, and exact versions of NumPy, pandas, Matplotlib, kagglehub and Stable-Baselines3.

Chapter 12

Future Enhancements

The following enhancements are proposals for future work. They were not implemented or evaluated in the current notebook.

1. Calibrate weather-dependent action uncertainty using measured trajectories or control-error observations rather than an assumed weather mapping.
2. Investigate alternative obstacle-distance thresholds and report sensitivity of realised map density and policy performance to the threshold.
3. Generate multiple obstacle maps and evaluate each trained method across a larger set of map seeds to assess spatial generalisation.
4. Increase the number of training seeds and use confidence intervals or suitable statistical tests to assess whether differences in mean success rate are reliable.
5. Introduce richer navigation states, including orientation, velocity, sensor observations and dynamic obstacles, while maintaining a clearly specified transition model.
6. Compare equal environment-step budgets with equal wall-clock or compute budgets to separate sample efficiency from computational efficiency.
7. Conduct systematic hyperparameter tuning for learning rates, exploration schedules, replay capacity, network sizes, PPO rollout length and entropy coefficient.
8. Add ablation experiments to isolate the contribution of dataset-informed obstacle density and weather-dependent slip from the baseline simulator.
9. Explore curriculum learning or progressively harder maps to examine whether the agents can learn robust policies under increasing obstacle complexity.
10. Investigate sequence-modelling approaches such as Decision Transformer as a separate extension, with appropriate trajectory data and a clearly documented training protocol.

Conclusion

The project successfully implemented a dataset-informed grid-world navigation experiment and evaluated the performance of four reinforcement learning algorithms: Q-Learning, SARSA, Deep Q-Network (DQN), and Proximal Policy Optimization (PPO). All algorithms were compared under a common training budget of 60,000 steps and three random seeds to ensure a relatively fair comparison. The dataset was used to determine obstacle density and episode-level slip distribution, while the simulator maintained project-defined transition and reward rules.

The experimental results showed that SARSA achieved the highest mean greedy evaluation success rate of **82.7%**, narrowly outperforming DQN, which achieved **81.0%**. This indicates that SARSA was slightly more reliable in successfully reaching the target under the given environment conditions. However, DQN achieved the highest mean reward and the shortest successful-route length, suggesting that it finds more efficient routes when it succeeds. The tabular approaches (Q-Learning and SARSA) also required considerably less training time compared to the deep reinforcement learning methods.

The comparison highlights that no single algorithm performed best across every evaluation metric. While SARSA provided the highest success rate, DQN demonstrated better performance in terms of reward and route efficiency. This shows that the choice of reinforcement learning algorithm depends on the specific objectives of the navigation task, such as reliability, efficiency, and computational requirements.

Although the experiment provided useful insights, certain limitations must be considered. The evaluation was conducted using only three random seeds, and the results were influenced by the assumptions and simplified conditions of the simulated environment. Further improvements could include testing with more random seeds, increasing the training budget, experimenting with different obstacle configurations, and evaluating the algorithms in more complex environments.

Overall, the project demonstrates the practical application of reinforcement learning techniques for autonomous grid-world navigation and provides a comparative understanding of model-free reinforcement learning algorithms. The findings offer a useful foundation for further exploration of intelligent navigation systems. However, all conclusions are restricted to the simulated task and should not be directly generalized to real-world robotic navigation without additional validation.

Bibliography

[1] Sutton, R.S. and Barto, A.G., 2018. Reinforcement Learning: An Introduction. 2nd ed. Cambridge, MA: MIT Press.

[2] Watkins, C.J.C.H. and Dayan, P., 1992. Q-learning. Machine Learning, 8(3–4), pp. 279–292.

[3] Rummery, G.A. and Niranjan, M., 1994. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166. Cambridge University Engineering Department.

[4] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G. and Petersen, S., 2015. Human-level control through deep reinforcement learning. Nature, 518(7540), pp. 529–533.

[5] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

[6] Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A. and Mordatch, I., 2021. Decision Transformer: Reinforcement learning via sequence modeling. Advances in Neural Information Processing Systems, 34.

[7] freshersstaff, n.d. Autonomous Navigation Control Dataset (version 2). Kaggle. Available at: https://www.kaggle.com/datasets/freshersstaff/autonomous-navigation-control-dataset.

[8] Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A. and Perez-Vicente, R., 2024. Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032.

[9] Lin, L.-J., 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. Machine Learning, 8(3–4), pp. 293–321.

[10] Kingma, D.P. and Ba, J., 2015. Adam: A method for stochastic optimization. In: International Conference on Learning Representations (ICLR). arXiv:1412.6980.

[11] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N., 2021. Stable-Baselines3: Reliable reinforcement learning implementations. Journal of Machine Learning Research, 22(268), pp. 1–8.

[12] Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P., 2016. High-dimensional continuous control using generalized advantage estimation. In: International Conference on Learning Representations (ICLR). arXiv:1506.02438.

[13] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. and Desmaison, A., 2019. PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

Appendix A

Source Code

This appendix reproduces every code cell in the uploaded notebook rl_riven_navigation_wid_op.ipynb in notebook order. The code cell source is included without rewriting or omission. Markdown cells and saved outputs are not source code and are therefore not reproduced here. The source text is preserved; long lines wrap naturally at the page boundary.

```
# ---------------- Cell 1: installs, imports, settings ----------------
import importlib, subprocess, sys
def ensure(pip_name, import_name=None):
    try:
        importlib.import_module(import_name or pip_name)
    except ImportError:
        print(f"Installing {pip_name} ...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", pip_name])

ensure("gymnasium"); ensure("stable-baselines3", "stable_baselines3"); ensure("kagglehub")

import os, re, glob, time, json, random, warnings
from collections import deque
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import torch, torch.nn as nn, torch.optim as optim
import gymnasium as gym
from gymnasium import spaces
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import BaseCallback
warnings.filterwarnings("ignore")

# ------------------------------ SETTINGS ------------------------------
QUICK_MODE = False                      # True = tiny debug run, False = full experiment
TRAIN_STEPS = 8_000 if QUICK_MODE else 60_000
SEEDS = [0] if QUICK_MODE else [0, 1, 2]
N_EVAL_EPISODES = 20 if QUICK_MODE else 100
GRID_SIZE = 8
MAX_EPISODE_STEPS = 100
MAP_SEED = 123
KAGGLE_DATASET = "freshersstaff/autonomous-navigation-control-dataset"
DATA_FILE = None                        # set a CSV path here to skip the Kaggle download
OBSTACLE_DIST_THRESHOLD_M = 15.0        # ASSUMPTION: an obstacle counts as "close" below this distance
USE_GPU_FOR_DEEP_RL = False
# ----------------------------------------------------------------------

def set_seed(seed):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)
set_seed(42)
DEVICE = torch.device("cuda" if (torch.cuda.is_available() and USE_GPU_FOR_DEEP_RL) else "cpu")
print("torch", torch.__version__, "| gymnasium", gym.__version__, "| device:", DEVICE)
print(f"QUICK_MODE={QUICK_MODE}, TRAIN_STEPS={TRAIN_STEPS}, SEEDS={SEEDS},
N_EVAL_EPISODES={N_EVAL_EPISODES}")
```

```
# ---------------- Cell 2: load the Kaggle driving dataset ----------------
if DATA_FILE:
    data_path = DATA_FILE
else:
    import kagglehub
    folder = kagglehub.dataset_download(KAGGLE_DATASET)
    files = glob.glob(os.path.join(folder, "**", "*.csv"), recursive=True)
    if not files: raise FileNotFoundError("No CSV found in the downloaded dataset.")
    data_path = max(files, key=os.path.getsize)
print("Using file:", data_path)

df = pd.read_csv(data_path).drop_duplicates().reset_index(drop=True)
print("Shape:", df.shape)
display(df.head())

def find_col(name_part):
    hits = [c for c in df.columns if name_part.lower() in c.lower()]
    if not hits: raise KeyError(f"No column containing '{name_part}'. Columns: {list(df.columns)}")
```

```
    return hits[0]

COL_DIST = find_col("distance_to_obstacle")
COL_WEATHER = find_col("weather")
print("Columns used:", COL_DIST, "|", COL_WEATHER)
```

```
# ---------------- Cell 3: derive the simulator parameters from the dataset ----------------
# (a) Obstacle density: share of driving situations in which an obstacle is closer than the threshold.
raw_density = float((df[COL_DIST] < OBSTACLE_DIST_THRESHOLD_M).mean())
OBSTACLE_DENSITY = float(np.clip(raw_density, 0.05, 0.25))   # clipped so the grid stays solvable and not
empty
print(f"Share of rows with {COL_DIST} < {OBSTACLE_DIST_THRESHOLD_M} m : {raw_density:.3f}")
print(f"Obstacle density used in the simulator              : {OBSTACLE_DENSITY:.3f}")

# (b) Slip noise from weather. The slip value per weather type is OUR ASSUMPTION (bad weather = less
reliable control).
SLIP_BY_WEATHER = {"clear": 0.05, "sunny": 0.05, "cloudy": 0.07, "overcast": 0.07,
                   "rainy": 0.10, "rain": 0.10, "foggy": 0.12, "fog": 0.12,
                   "snowy": 0.15, "snow": 0.15}
DEFAULT_SLIP = 0.10
def slip_of(w):
    return SLIP_BY_WEATHER.get(str(w).strip().lower(), DEFAULT_SLIP)

weather_table = df[COL_WEATHER].value_counts().rename("rows").to_frame()
weather_table["share"] = (weather_table["rows"] / len(df)).round(3)
weather_table["slip_probability"] = [slip_of(w) for w in weather_table.index]
weather_table["mapped"] = ["yes" if str(w).strip().lower() in SLIP_BY_WEATHER else "NO (default used)" for w
in weather_table.index]
display(weather_table)

# One slip value per dataset row. Each episode will sample one of them.
SLIP_VALUES = np.array([slip_of(w) for w in df[COL_WEATHER]], dtype=float)
print(f"Average slip probability over the dataset: {SLIP_VALUES.mean():.3f}")
```

```
# ---------------- Cell 4: the dataset-driven navigation environment ----------------
class NavigationEnv(gym.Env):
    ACTIONS = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}      # up, down, left, right as (row, col)
    R_GOAL, R_COLLISION, R_WALL, R_STEP = 100.0, -50.0, -3.0, -1.0

    def __init__(self, grid_size=GRID_SIZE, obstacle_density=OBSTACLE_DENSITY, map_seed=MAP_SEED,
                 slip_values=SLIP_VALUES, slip_override=None, max_steps=MAX_EPISODE_STEPS):
        super().__init__()
        self.n, self.max_steps = grid_size, max_steps
        self.slip_values, self.slip_override = np.asarray(slip_values, float), slip_override
        self.start, self.goal = (0, 0), (grid_size - 1, grid_size - 1)
        self.obstacles = self._make_map(obstacle_density, map_seed)
        self.action_space = spaces.Discrete(4)
        self.observation_space = spaces.Box(low=-1.0, high=1.0, shape=(8,), dtype=np.float32)
        self.pos, self.steps, self.cur_slip = self.start, 0, 0.0

    def _solvable(self, obs):                                      # breadth-first search
        seen, queue = {self.start}, deque([self.start])
        while queue:
            r, c = queue.popleft()
            if (r, c) == self.goal: return True
            for dr, dc in self.ACTIONS.values():
                nr, nc = r + dr, c + dc
                if 0 <= nr < self.n and 0 <= nc < self.n and not obs[nr, nc] and (nr, nc) not in seen:
                    seen.add((nr, nc)); queue.append((nr, nc))
        return False

    def _make_map(self, density, map_seed):
        rng = np.random.default_rng(map_seed)
        while True:
            obs = rng.random((self.n, self.n)) < density
            obs[self.start] = False; obs[self.goal] = False
            if self._solvable(obs): return obs

    def _blocked(self, r, c):
        return (not (0 <= r < self.n and 0 <= c < self.n)) or bool(self.obstacles[r, c])

    def _obs(self):
        r, c = self.pos; m = self.n - 1
        sensors = [float(self._blocked(r + dr, c + dc)) for dr, dc in self.ACTIONS.values()]
        return np.array([r / m, c / m, (self.goal[0] - r) / m, (self.goal[1] - c) / m] + sensors,
dtype=np.float32)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
```

```
        self.pos, self.steps = self.start, 0
        # DATASET LINK: this episode's slip probability comes from a randomly drawn dataset row (its
weather).
        self.cur_slip = self.slip_override if self.slip_override is not None else
float(self.np_random.choice(self.slip_values))
        return self._obs(), {"slip": self.cur_slip}

    def step(self, action):
        if self.np_random.random() < self.cur_slip:
            action = int(self.np_random.integers(4))
        dr, dc = self.ACTIONS[int(action)]
        nr, nc = self.pos[0] + dr, self.pos[1] + dc
        reward, terminated, collision, success = self.R_STEP, False, False, False
        if not (0 <= nr < self.n and 0 <= nc < self.n):
            reward = self.R_WALL                                    # wall bump: stay in place
        elif self.obstacles[nr, nc]:
            reward, terminated, collision = self.R_COLLISION, True, True
        else:
            self.pos = (nr, nc)
            if self.pos == self.goal:
                reward, terminated, success = self.R_GOAL, True, True
        self.steps += 1
        truncated = (not terminated) and self.steps >= self.max_steps
        return self._obs(), reward, terminated, truncated, {"is_success": success, "collision": collision}

def make_env(**kw): return NavigationEnv(**kw)

def discretize(obs, n=GRID_SIZE):                                   # observation -> grid-cell id (for Q-
tables)
    return int(round(float(obs[0]) * (n - 1))) * n + int(round(float(obs[1]) * (n - 1)))

env_demo = make_env()
print("Obstacle cells:", int(env_demo.obstacles.sum()), "of", GRID_SIZE * GRID_SIZE)
print("Obstacle positions (row, col):", [tuple(map(int, x)) for x in np.argwhere(env_demo.obstacles)])
o, info = env_demo.reset(seed=0)
print("First observation:", np.round(o, 2), "| slip this episode:", info["slip"])

def draw_map(ax, env, title, path=None):
    ax.imshow(np.where(env.obstacles, 1.0, 0.0), cmap="Greys", vmin=0, vmax=1.6)
    ax.text(env.start[1], env.start[0], "S", ha="center", va="center", color="#0072B2", fontsize=14,
fontweight="bold")
    ax.text(env.goal[1], env.goal[0], "G", ha="center", va="center", color="#009E73", fontsize=14,
fontweight="bold")
    if path: ax.plot([p[1] for p in path], [p[0] for p in path], "-o", color="#D55E00", ms=4, lw=2)
    ax.set_xticks(range(env.n)); ax.set_yticks(range(env.n)); ax.set_title(title)
    ax.set_xticks(np.arange(-.5, env.n, 1), minor=True); ax.set_yticks(np.arange(-.5, env.n, 1), minor=True)
    ax.grid(which="minor", color="lightgray", lw=0.5); ax.tick_params(which="minor", length=0)

fig, ax = plt.subplots(figsize=(4.5, 4.5))
draw_map(ax, env_demo, "Dataset-driven map (black = obstacle)"); plt.show()
```

```
# ---------------- Cell 5: shared training / evaluation utilities ----------------
GAMMA = 0.99
EPS_START, EPS_END, EPS_FRACTION = 1.0, 0.05, 0.6

def epsilon_at(step, total=TRAIN_STEPS):
    return EPS_START + min(1.0, step / (EPS_FRACTION * total)) * (EPS_END - EPS_START)

def new_log(): return {"end_step": [], "reward": [], "success": [], "collision": [], "length": []}

def add_episode(log, step, ep_reward, info, length):
    log["end_step"].append(step); log["reward"].append(ep_reward)
    log["success"].append(int(info.get("is_success", False)))
    log["collision"].append(int(info.get("collision", False))); log["length"].append(length)

class GreedyAgent:
    def __init__(self, act_fn): self.act = act_fn

def evaluate(agent, n_episodes=N_EVAL_EPISODES, seed_base=10_000):
    # Greedy evaluation on fixed seeds. Slip is still sampled from the dataset's weather mix.
    env, rows = make_env(), []
    for i in range(n_episodes):
        obs, _ = env.reset(seed=seed_base + i)
        done, total, steps, info = False, 0.0, 0, {}
        while not done:
            obs, r, term, trunc, info = env.step(agent.act(obs))
            total += r; steps += 1; done = term or trunc
        rows.append({"reward": total, "success": int(info["is_success"]),
                     "collision": int(info["collision"]), "steps": steps})
```

```
    d = pd.DataFrame(rows)
    return {"avg_reward": d.reward.mean(), "success_rate": d.success.mean(),
            "avg_steps_to_goal": d.steps[d.success == 1].mean() if d.success.sum() else np.nan,
            "collision_rate": d.collision.mean(),
            "timeout_rate": 1 - d.success.mean() - d.collision.mean()}
print("Shared tools ready.")
```

```
# ---------------- Cell 6: Q-Learning and SARSA (tabular) ----------------
ALPHA = 0.1
N_STATES, N_ACTIONS = GRID_SIZE * GRID_SIZE, 4

def train_tabular(algo, seed, total_steps=TRAIN_STEPS):
    rng = np.random.default_rng(seed)
    Q = np.zeros((N_STATES, N_ACTIONS))
    env, log = make_env(), new_log()
    def choose(s, step):
        if rng.random() < epsilon_at(step, total_steps): return int(rng.integers(N_ACTIONS))
        return int(rng.choice(np.flatnonzero(Q[s] == Q[s].max())))       # random tie-breaking
    step, first = 0, True
    while step < total_steps:
        obs, _ = env.reset(seed=seed if first else None); first = False
        s = discretize(obs); a = choose(s, step)
        ep_r, ep_len, done, info = 0.0, 0, False, {}
        while not done and step < total_steps:
            obs2, r, term, trunc, info = env.step(a); step += 1
            s2 = discretize(obs2); a2 = choose(s2, step)
            if term: target = r
            elif algo == "qlearning": target = r + GAMMA * Q[s2].max()      # off-policy
            else: target = r + GAMMA * Q[s2, a2]                            # SARSA, on-policy
            Q[s, a] += ALPHA * (target - Q[s, a])
            s, a = s2, a2; ep_r += r; ep_len += 1; done = term or trunc
            if done: add_episode(log, step, ep_r, info, ep_len)
    return GreedyAgent(lambda o: int(Q[discretize(o)].argmax())), log

_a, _l = train_tabular("qlearning", seed=0, total_steps=500)
print("Sanity check OK - episodes logged:", len(_l["reward"]))
```

```
# ---------------- Cell 7: DQN ----------------
class QNetwork(nn.Module):
    def __init__(self, n_in=8, n_out=4, hidden=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, hidden), nn.ReLU(), nn.Linear(hidden, hidden), nn.ReLU(),
nn.Linear(hidden, n_out))
    def forward(self, x): return self.net(x)

class ReplayBuffer:
    def __init__(self, capacity=20_000, obs_dim=8):
        self.cap, self.i, self.size = capacity, 0, 0
        self.s = np.zeros((capacity, obs_dim), np.float32); self.s2 = np.zeros((capacity, obs_dim),
np.float32)
        self.a = np.zeros(capacity, np.int64); self.r = np.zeros(capacity, np.float32); self.term =
np.zeros(capacity, np.float32)
    def add(self, s, a, r, s2, term):
        self.s[self.i], self.a[self.i], self.r[self.i], self.s2[self.i], self.term[self.i] = s, a, r, s2,
float(term)
        self.i = (self.i + 1) % self.cap; self.size = min(self.size + 1, self.cap)
    def sample(self, batch, rng):
        idx = rng.integers(0, self.size, size=batch)
        t = lambda x: torch.as_tensor(x[idx], device=DEVICE)
        return t(self.s), t(self.a), t(self.r), t(self.s2), t(self.term)

DQN_LR, DQN_BATCH, DQN_LEARN_START, DQN_TARGET_EVERY = 1e-3, 64, 500, 250

def train_dqn(seed, total_steps=TRAIN_STEPS):
    set_seed(seed); rng = np.random.default_rng(seed)
    q, q_target = QNetwork().to(DEVICE), QNetwork().to(DEVICE)
    q_target.load_state_dict(q.state_dict())
    opt, buf, loss_fn = optim.Adam(q.parameters(), lr=DQN_LR), ReplayBuffer(), nn.SmoothL1Loss()
    env, log = make_env(), new_log()
    def greedy(o):
        with torch.no_grad():
            return int(q(torch.as_tensor(o, dtype=torch.float32,
device=DEVICE).unsqueeze(0)).argmax(1).item())
    step, first = 0, True
    while step < total_steps:
        obs, _ = env.reset(seed=seed if first else None); first = False
        ep_r, ep_len, done, info = 0.0, 0, False, {}
        while not done and step < total_steps:
            a = int(rng.integers(4)) if rng.random() < epsilon_at(step, total_steps) else greedy(obs)
```

```
            obs2, r, term, trunc, info = env.step(a); step += 1
            buf.add(obs, a, r, obs2, term)                                  # only true termination stops
bootstrapping
            obs, ep_r, ep_len, done = obs2, ep_r + r, ep_len + 1, term or trunc
            if step >= DQN_LEARN_START and buf.size >= DQN_BATCH:
                s, a_b, r_b, s2, t_b = buf.sample(DQN_BATCH, rng)
                q_sa = q(s).gather(1, a_b.unsqueeze(1)).squeeze(1)
                with torch.no_grad():
                    y = r_b + GAMMA * (1 - t_b) * q_target(s2).max(1)[0]
                loss = loss_fn(q_sa, y)
                opt.zero_grad(); loss.backward()
                nn.utils.clip_grad_norm_(q.parameters(), 10.0); opt.step()
            if step % DQN_TARGET_EVERY == 0: q_target.load_state_dict(q.state_dict())
            if done: add_episode(log, step, ep_r, info, ep_len)
    return GreedyAgent(greedy), log

_a, _l = train_dqn(seed=0, total_steps=1000)
print("Sanity check OK - episodes logged:", len(_l["reward"]))
```

```
# ---------------- Cell 8: PPO (Stable-Baselines3) ----------------
class EpisodeLogger(BaseCallback):
    def __init__(self):
        super().__init__(); self.log, self.ep_r, self.ep_len = new_log(), 0.0, 0
    def _on_step(self):
        self.ep_r += float(self.locals["rewards"][0]); self.ep_len += 1
        if bool(self.locals["dones"][0]):
            add_episode(self.log, self.num_timesteps, self.ep_r, self.locals["infos"][0], self.ep_len)
            self.ep_r, self.ep_len = 0.0, 0
        return True

PPO_N_STEPS, PPO_BATCH = 500, 100

def train_ppo(seed, total_steps=TRAIN_STEPS):
    set_seed(seed)
    env = make_env()
    model = PPO("MlpPolicy", env, n_steps=PPO_N_STEPS, batch_size=PPO_BATCH, n_epochs=10,
                learning_rate=3e-4, gamma=GAMMA, gae_lambda=0.95, clip_range=0.2, ent_coef=0.01,
                policy_kwargs=dict(net_arch=[64, 64]), seed=seed, device=str(DEVICE), verbose=0)
    cb = EpisodeLogger()
    model.learn(total_timesteps=total_steps, callback=cb)
    return GreedyAgent(lambda o: int(model.predict(o, deterministic=True)[0])), cb.log

_a, _l = train_ppo(seed=0, total_steps=1000)
print("Sanity check OK - episodes logged:", len(_l["reward"]))
```

```
# ---------------- Cell 9: train all 4 algorithms x all seeds ----------------
ALGOS = {"Q-Learning": lambda s: train_tabular("qlearning", s),
         "SARSA":      lambda s: train_tabular("sarsa", s),
         "DQN":        lambda s: train_dqn(s),
         "PPO":        lambda s: train_ppo(s)}
results, agents = {a: {} for a in ALGOS}, {a: {} for a in ALGOS}
for name, fn in ALGOS.items():
    for seed in SEEDS:
        t0 = time.time()
        agent, log = fn(seed)
        results[name][seed] = {"log": log, "train_time": time.time() - t0}
        agents[name][seed] = agent
        print(f"{name:10s} seed {seed}: {len(log['reward']):5d} episodes,
{results[name][seed]['train_time']:6.1f}s")
```

```
# ---------------- Cell 10: evaluation and metrics ----------------
def moving_avg(x, w):
    x = np.asarray(x, dtype=float)
    if len(x) == 0: return x
    w = min(w, len(x)); return np.convolve(x, np.ones(w) / w, mode="valid")

def convergence_step(log, window=50, thresh=0.8):
    ma = moving_avg(log["success"], window)
    idx = np.flatnonzero(ma >= thresh)
    return np.nan if len(idx) == 0 else log["end_step"][idx[0] + min(window, len(log["success"])) - 1]

def stability(log):
    r = [x for x, s in zip(log["reward"], log["end_step"]) if s >= 0.8 * TRAIN_STEPS]
    return float(np.std(r)) if len(r) > 1 else np.nan

for name in ALGOS:
    for seed in SEEDS:
        res = results[name][seed]
        res["eval"] = evaluate(agents[name][seed])
```

```
        res["convergence_step"] = convergence_step(res["log"])
        res["stability_std"] = stability(res["log"])

rows = []
for name in ALGOS:
    for seed in SEEDS:
        r = results[name][seed]
        rows.append({"algorithm": name, "seed": seed, **r["eval"], "convergence_step":
r["convergence_step"],
                     "stability_std": r["stability_std"], "train_time_s": r["train_time"]})
per_seed = pd.DataFrame(rows)
print("Per-seed results:")
display(per_seed.round(3))
```

```
# ---------------- Cell 11: comparison table (mean +/- std over seeds) ----------------
def pm(series, d=2):
    s = series.dropna()
    return "n/a" if len(s) == 0 else f"{s.mean():.{d}f} ± {s.std(ddof=0):.{d}f}"

table = []
for name in ALGOS:
    d = per_seed[per_seed.algorithm == name]
    table.append({"Algorithm": name, "Avg reward": pm(d.avg_reward, 1), "Success rate": pm(d.success_rate,
3),
                  "Avg steps to goal": pm(d.avg_steps_to_goal, 1), "Collision rate": pm(d.collision_rate,
3),
                  "Convergence step": pm(d.convergence_step, 0) + f"
({int(d.convergence_step.notna().sum())}/{len(d)} seeds)",
                  "Stability (reward std)": pm(d.stability_std, 1), "Train time (s)": pm(d.train_time_s,
1)})
comparison = pd.DataFrame(table).set_index("Algorithm")
display(comparison)

means = per_seed.groupby("algorithm")[["avg_reward", "success_rate", "avg_steps_to_goal", "collision_rate",
                                       "convergence_step", "stability_std", "train_time_s"]].mean()
medians = per_seed.groupby("algorithm")[["success_rate", "avg_reward"]].median()
print("\nMedian over seeds (less affected by one bad seed):")
display(medians.round(3))

os.makedirs("outputs", exist_ok=True)
per_seed.to_csv("outputs/per_seed_results.csv", index=False)
means.to_csv("outputs/mean_results.csv")
with open("outputs/training_logs.json", "w") as f:
    json.dump({a: {str(s): results[a][s]["log"] for s in SEEDS} for a in ALGOS}, f)
print("Saved to outputs/: per_seed_results.csv, mean_results.csv, training_logs.json")
```

```
# ---------------- Cell 12: training curves ----------------
COLORS = {"Q-Learning": "#0072B2", "SARSA": "#E69F00", "DQN": "#009E73", "PPO": "#CC79A7"}
WINDOW = 50
seed0 = SEEDS[0]
fig, axes = plt.subplots(2, 2, figsize=(13, 8), sharey=True)
for ax, name in zip(axes.ravel(), ALGOS):
    lg = results[name][seed0]["log"]
    ax.plot(lg["end_step"], lg["reward"], color=COLORS[name], alpha=0.25, lw=0.8, label="episode reward
(raw)")
    ma = moving_avg(lg["reward"], WINDOW)
    ax.plot(lg["end_step"][len(lg["end_step"]) - len(ma):], ma, color=COLORS[name], lw=2, label=f"{WINDOW}-
episode average")
    ax.set_title(f"{name} - training reward (seed {seed0})")
    ax.set_xlabel("environment steps"); ax.set_ylabel("episode reward"); ax.legend(loc="lower right")
plt.tight_layout(); plt.show()

grid = np.linspace(TRAIN_STEPS * 0.02, TRAIN_STEPS, 200)
def curve_on_grid(log, key):
    ma = moving_avg(log[key], WINDOW)
    xs = np.asarray(log["end_step"][len(log["end_step"]) - len(ma):], dtype=float)
    return np.interp(grid, xs, ma) if len(xs) > 1 else np.full_like(grid, np.nan)

for key, ylabel, title in [("reward", "average episode reward", "Smoothed training reward (mean ± std over
seeds)"),
                           ("success", "training success rate", "Smoothed training success rate (mean ± std
over seeds)")]:
    plt.figure(figsize=(9, 5))
    for name in ALGOS:
        cs = np.array([curve_on_grid(results[name][s]["log"], key) for s in SEEDS])
        m, sd = np.nanmean(cs, 0), np.nanstd(cs, 0)
        plt.plot(grid, m, color=COLORS[name], lw=2, label=name)
        plt.fill_between(grid, m - sd, m + sd, color=COLORS[name], alpha=0.15)
    plt.xlabel("environment steps"); plt.ylabel(ylabel); plt.title(title)
```

```
    plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
print("Note: training curves include exploration, so they lie below the greedy evaluation results.")
```

```
# ---------------- Cell 13: bar charts, cost plot and greedy paths ----------------
metrics = [("success_rate", "Success rate", "fraction of episodes"),
           ("avg_steps_to_goal", "Average steps to goal (successful episodes)", "steps"),
           ("collision_rate", "Collision rate", "fraction of episodes"),
           ("avg_reward", "Average cumulative reward", "reward per episode")]
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, (col, title, yl) in zip(axes.ravel(), metrics):
    g = per_seed.groupby("algorithm")[col]
    m = g.mean().reindex(list(ALGOS)); sd = g.std(ddof=0).reindex(list(ALGOS))
    ax.bar(m.index, m.values, yerr=sd.values, capsize=4, color=[COLORS[a] for a in m.index],
edgecolor="white")
    ax.set_title(title); ax.set_ylabel(yl); ax.set_xlabel("algorithm"); ax.grid(axis="y", alpha=0.3)
plt.suptitle("Greedy evaluation results (error bars = std over seeds)")
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(8, 5))
for i, name in enumerate(ALGOS):
    d = per_seed[per_seed.algorithm == name]
    ax.errorbar(d.train_time_s.mean(), d.success_rate.mean(), yerr=d.success_rate.std(ddof=0), fmt="o",
ms=11,
                color=COLORS[name], capsize=4, label=name)
    ax.annotate(name, (d.train_time_s.mean(), d.success_rate.mean()), textcoords="offset points",
xytext=(10, 6 + 14 * i))
ax.set_xlabel("mean training time (seconds)"); ax.set_ylabel("mean evaluation success rate")
ax.set_ylim(-0.05, 1.05); ax.set_title("Performance vs training cost (error bars = std over seeds)")
ax.grid(alpha=0.3); ax.legend(loc="lower right"); plt.tight_layout(); plt.show()

# Greedy paths of the seed-0 agents (slip switched OFF only for this illustration)
fig, axes = plt.subplots(1, 4, figsize=(16, 4.3))
for ax, name in zip(axes, ALGOS):
    e = make_env(slip_override=0.0); o, _ = e.reset(seed=0); path = [e.pos]; done = False
    while not done:
        o, _, te, tr, _ = e.step(agents[name][seed0].act(o)); path.append(e.pos); done = te or tr
    draw_map(ax, e, f"{name}: greedy path ({len(path)-1} steps)", path)
plt.tight_layout(); plt.show()
```

```
# ---------------- Cell 14: ranking and honest summary ----------------
ranking = means.sort_values(["success_rate", "avg_reward", "collision_rate"], ascending=[False, False,
True])
best = ranking.index[0]
rank_table = ranking[["success_rate", "avg_reward", "collision_rate", "avg_steps_to_goal",
"train_time_s"]].round(3)
rank_table.insert(0, "rank", range(1, len(rank_table) + 1))
display(rank_table)

sd = per_seed.groupby("algorithm").success_rate.std(ddof=0)
second = ranking.index[1]
gap = ranking.loc[best, "success_rate"] - ranking.loc[second, "success_rate"]
print(f"Best by mean success rate: {best}. Lead over {second}: {gap:.3f} (seed std: {sd[best]:.3f} vs
{sd[second]:.3f})")
print("-> the lead is", "larger than" if gap > max(sd[best], sd[second]) else "NOT clearly larger than",
      "the seed-to-seed variation; with", len(SEEDS), "seed(s) treat this as indicative, not proof.")
print(f"Cheapest to train: {means.train_time_s.idxmin()} ({means.train_time_s.min():.0f}s); "
      f"most expensive: {means.train_time_s.idxmax()} ({means.train_time_s.max():.0f}s).")
print("\nSimulator parameters that came from the Kaggle dataset:")
print(f"  obstacle density = {OBSTACLE_DENSITY:.3f}  (share of rows with {COL_DIST} <
{OBSTACLE_DIST_THRESHOLD_M} m)")
print(f"  slip probability = sampled per episode from dataset weather, mean {SLIP_VALUES.mean():.3f}")
if QUICK_MODE: print("\nWARNING: QUICK_MODE is on - do not draw conclusions from these numbers.")
print("All results come from a SIMULATED environment, not a real vehicle.")
```

```
# ---------------- Success Rate Comparison - Highlight Best Model ----------------

import matplotlib.pyplot as plt

success_rates = ranking["success_rate"].copy()

# Convert to percentage if values are between 0 and 1
if success_rates.max() <= 1:
    success_rates = success_rates * 100

# Identify the best model
best_model = success_rates.idxmax()

# Highlight best model
colors = [
```

```
    "#16A085" if model == best_model else "#AAB7B8"
    for model in success_rates.index
]

plt.figure(figsize=(9, 5))

bars = plt.bar(
    success_rates.index.astype(str),
    success_rates.values,
    color=colors,
    edgecolor="black",
    width=0.6
)

plt.title(
    f"Success Rate Comparison - Best Model: {best_model}",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("RL Algorithms", fontsize=12)
plt.ylabel("Success Rate (%)", fontsize=12)

plt.ylim(0, 100)

plt.bar_label(
    bars,
    labels=[f"{v:.2f}%" for v in success_rates.values],
    padding=4,
    fontweight="bold"
)

plt.grid(axis="y", linestyle="--", alpha=0.4)

plt.tight_layout()
plt.show()

print(f"Highest Success Rate Model: {best_model}")
print(f"Success Rate: {success_rates[best_model]:.2f}%")
```